

CS336 第5次作业（对齐）：对齐与推理 RL

1.0.2版

CS336 Staff

2025年春季

1 任务概述

本任务将带您亲身体验训练语言模型解决数学问题时的推理能力。

您将要实施的措施。

1. 竞赛数学问题MATH数据集的零样本提示基线Hendrycks等人[2021]。
2. 监督微调，基于更强推理模型（DeepSeek R1，DeepSeek AI等2025）的推理轨迹。
3. 通过验证奖励提升推理性能的专家迭代方法
4. 基于验证奖励的群体相对策略优化（GRPO）技术，用于提升推理性能。

对于感兴趣的人，我们将有一个**完全可选**的任务部分，即对语言模型与人类偏好的对齐，这将在未来几天发布。

你将要运行的程序。

1. 测量Qwen 2.5数学1.5B零样本提示性能（本研究基准）。
2. 在Qwen 2.5 Math 1.5B模型上运行 SFT，同时保留R1阶段的推理轨迹。
3. 在Qwen 2.5 Math 1.5B模型上运行专家迭代，并采用经过验证的奖励机制。
4. 在Qwen 2.5数学1上运行 GRPO。5B，奖励已验证。

代码的具体形式。所有作业代码及本说明文档均可在 GitHub 获取，地址为：github.com/stanford-cs336/assignment5-alignment

请通过git克隆该仓库。如有更新，我们将通知您，您可执行git pull获取最新版本。

1. cs336_alignment /*：此处是编写第五次作业代码的地方。请注意此处没有现成代码（除少量入门代码外），因此你可以从零开始自由编写。

2. `cs336_alignment /提示/*`: 为方便操作, 我们已提供文本文件中的提示, 以减少从PDF复制粘贴提示到代码时可能产生的错误。
3. `tests/* .py`: 此文件包含所有必须通过的测试。**您只需通过测试/test_sft.py 和 tests/test-grpo.py——其余测试用于非必修部分的作业。**这些测试调用 `tests/adapters.py` 中定义的钩子。您需要实现适配器以将代码连接到测试。编写更多测试和/或修改测试代码有助于调试代码, 但您的实现需通过原始提供的测试套件。
4. `readme.md`: 此文件包含一些关于设置环境的基本说明。

可用工具。我们要求你从零开始构建大部分强化学习相关组件。你可以使用vLLM等工具从语言模型生成文本 (§3.1)。此外, 您可以使用HuggingFace Transformers加载Qwen 2.5 Math 1.5B模型和分词器并运行前向传递 (§4.1), 但不得使用任何训练工具 (例如Trainer类)。

如何提交。请将以下文件提交至Gradescope:

- `writeup.pdf`: 请回答所有书面问题。请将您的回答排版。
- `code.zip`: 包含您编写的所有代码。

2 语言模型推理

2.1 动机

语言模型最突出的应用场景之一, 是构建能够处理多种自然语言处理任务的通用系统。本课题将聚焦于语言模型的一个新兴应用场景——数学推理。该场景将作为实验平台, 用于建立评估体系、实施监督微调, 并通过强化学习 (RL) 技术对语言模型进行推理能力训练。

这次作业将与我们以往的作业方式有所不同, 主要体现在两个方面。

- 首先, 我们不会使用之前的语言模型代码库和模型。我们原本希望使用基于先前任务训练的基础语言模型, 但微调这些模型效果不佳——它们的数学推理能力实在有限。因此, 我们决定采用现成的高性能语言模型 (Qwen 2.5 Math 1.5B Base), 并以此为基础开展大部分工作。
- 其次, 我们将引入一个新基准来评估语言模型。此前我们一直认为交叉熵是许多下游任务的良好替代指标。但本次任务的核心目标在于弥合基础模型与下游任务之间的差距, 因此必须采用独立于交叉熵的评估方法。我们将使用Hendrycks等人开发的MATH 12K数据集。[2021]该系统包含具有挑战性的高中竞赛数学题目。我们将通过将语言模型输出与参考答案进行比对来评估其性能。

2.2 链式推理与推理强化学习

语言模型最近的一个令人兴奋的趋势是使用*思维链*推理来提高各种任务的性能。思维链指的是通过逐步推理解决问题的过程, 在得出最终答案之前生成中间推理步骤。

使用大语言模型进行思维链推理。早期的思维链方法通过使用“草稿本”将问题分解为中间步骤，对语言模型进行微调以解决算术等简单数学任务[Nye等人，2021]。其他研究则提示强模型在回答前“逐步思考”，发现这能显著提升小学数题等数学推理任务的性能[Wei等人，2023]。

通过专家迭代学习推理。自学推理器（STaR）[Zelikman等人，2022]将推理框架为引导循环：预训练模型首先采样多样化的思维链（CoTs），仅保留那些导向正确答案的路径，然后在这些“专家”轨迹上进行微调。迭代此循环可提升语言模型的推理能力和解题效率。STaR证明了这种专家迭代版本[Anthony等人，2017年]通过基于自动字符串匹配的生成答案验证，可在无需人工编写推理轨迹的情况下提升推理能力。

使用验证奖励的推理强化学习，o1与R1。近期研究探索了采用更强大的强化学习算法配合验证奖励来提升推理性能。OpenAI的o1（及后续的o3/o4）[OpenAI等人，2024]、DeepSeek的R1[DeepSeek-AI等人，2025]以及Moonshot的kimi k1.5[Team等人，2025]均采用策略梯度方法[Sutton等人，1999]进行数学与编程任务训练，其中字符串匹配或单元测试验证正确性，展现出在竞赛数学与编程表现上的显著提升。后续研究如Open-R1[Face，2025]、SimpleRL-Zoo[Zeng等人，2025]和TinyZero[Pan等人，2025]证实，即使在模型规模小至1的场景下，采用验证奖励的纯强化学习仍能取得显著效果。5B参数——可提升推理性能。

我们的实验设置：模型与数据集。在后续章节中，我们将逐步采用更复杂的训练方法，通过训练基础语言模型实现分步推理以解决数学问题。本任务将使用Qwen 2.5 Math 1.5B基础模型，该模型基于高质量合成数学预训练数据[Yang等人，2024]，从Qwen 2.5 1.5B模型持续预训练而来。MATH数据集可在Together集群的/data/a5-alignment/MATH路径获取。

开源审计师的提示:替代数据集

遗憾的是，由于版权问题，MATH数据集无法公开获取。若您在家自行学习，可选用以下开源数学推理数据集之一：

- Countdown[Pan等人，2025]，可在此处获取：一个基于英国电视节目Countdown的简单合成任务，该任务作为小规模推理强化学习的流行测试平台。
- GSM8K [Cobbe等人，2021a，可在此处获取：小学数学问题，这些问题比MATH更简单，但应能让你调试正确性并熟悉推理RL流程。
- Tulu 3 SFT Math [Lambert等人，2025]，可在此获取：使用GPT-4o和Claude 3.5 Sonnet生成的合成数学问题。由于这些是合成的，某些答案（甚至问题本身）可能并不完全正确。
- 其他一些数学 SFT 数据集链接[这里](#)。

若未直接提供短格式真实标签（如1/2），可使用Math-Verify等数学解析工具处理真实标签列。

3 零射击数学成绩的测量

我们将首先在MATH的5K示例测试集上评估基础语言模型的性能。建立这一基准有助于理解后续每种方法如何影响模型行为。

除非另有说明，我们在MATH实验中将使用DeepSeek R1-Zero模型[DeepSeek-AI等人，2025]的以下提示。我们将此称为 `r1_zero` 提示：

用户与助手的对话。用户提出问题，助手 `<-`解决问题。助手首先在脑海中思考推理过程，`<-`然后向用户给出答案。推理过程分别用 `<-(think) (/think)` 标记，答案用 `(answer) (/answer)` 标记，`<-`即 `(think)` 推理过程在此 `(/think) (answer)` 答案在此 `(/answer)`。

User: {question}

Assistant: `<think>`

`r1_zero` 提示位于文本文件 `cs336_alignment /prompts/ r1_zero .prompt`中。

在提示中，问题指的是我们插入的某个问题（例如：娜塔莉亚在四月向48位朋友出售了夹子，随后在五月出售了数量减半的夹子。娜塔莉亚在四月和五月总共售出了多少夹子？）。该模型的预期功能是充当辅助工具，首先通过添加左思考标签 `(think)` 启动思维过程，随后用 `(/think)` 结束该过程，并在答案标签内生成最终符号化回答，例如 `(answer) 4x + 10 (/answer)`。设置 `(answer) (/answer)` 这类标签的目的是为了便于我们解析模型输出并与标准答案进行比对，同时确保在检测到正确答案标签时即可停止生成响应。

提示选择说明。研究发现，`r1_zero` 提示并非最大化效果的最佳选择

由于提示词与Qwen 2.5数学模型的处理方式不匹配，导致其在强化学习（RL）后的下游性能表现不佳。1.5B模型已完成预训练。刘等人[2025] 研究发现，仅用问题（不添加其他内容）提示模型时，其初始准确率就非常高，例如在强化学习（RL）100多步后就能匹配 `r1_zero` 提示。他们的研究表明，Qwen2.5Math 1.5B已预先训练过此类问答对。

尽管如此，我们选择 `r1_zero` 提示作为本次作业，因为使用该提示进行强化学习（RL）能在短时间内显著提升准确率，这让我们能够快速掌握RL的运作机制并验证其正确性，即使最终表现不尽如人意。作为现实检验，你将在作业后期直接与`question_only`提示进行对比。

3.1 利用vLLM进行离线语言模型推理

为评估我们的语言模型，我们需要为多种提示生成连续响应（续写）。虽然可以像作业1那样自行开发生成函数，但强化学习（RL）的高效实现需要高性能推理技术，而这些技术的实现超出了本作业的范围。因此，本作业建议使用vLLM进行离线批量推理。vLLM是一款高吞吐量且内存高效的语言模型推理引擎，集成了多种实用的效率技术（例如优化的 CUDA 内核、用于高效注意力KV缓存的PagedAttention[Kwon等人，2023]等）。使用 vLLM 为提示列表生成续写内容：¹

来自vllm导入LLM，采样参数

示例提示。

```
prompts = [
    "Hello, my name is"
    ,
    "The president of the United States is"
    ,
    "The capital of France is"
    , "人工智能的未来是"
    ,
]
```

¹ 示例摘自https://github.com/vllm-project/vllm/blob/main/examples/offline_inference.py。

创建采样参数对象，停止在新行处生成。

```
sampling_params = SamplingParams(
    temperature=1.0, top_p=1.0, max_tokens=1024, stop= [ "\n" ])
```

创建一个LLM。

```
llm = LLM(model=<path to model>)
```

根据提示生成文本。输出结果为RequestOutput对象列表

包含提示、生成文本及其他信息的文件。

```
outputs = llm.generate(prompts, sampling_params)
```

打印输出结果。

用于输出在输出中：prompt =

```
output.prompt
```

```
generated_text = output.outputs[0].text
```

```
打印(f "提示: {prompt! r}, 生成文本: {generated_text! r}" )
```

在上述示例中，大语言模型（LLM）可通过HuggingFace模型名称（若本地未找到该模型将自动下载并缓存）或模型路径进行初始化。由于下载耗时较长（特别是参数量较大的模型，例如700亿参数），且为节省集群磁盘空间（避免每位用户都拥有独立的预训练模型副本），我们已在Together集群上下载了以下路径的预训练模型。**请不要在Together集群上重新下载这些模型：**

- Qwen 2.5数学1.5B基础版（推理 experiments):/data/a5-alignment/models/Qwen2.5-Math-1.5B)
- Llama 3.1 8B Base（支持可选指令调优 experiments):/data/a5-alignment/models/Llama-3.1-8B)
- Llama 3.3 70B指令（用于可选指令调优 experiments):/data/a5-alignment/models/Llama-3.3-70B-Instruct

3.2 零样本MATH基线

提示设置。为评估数学测试集上的零样本性能，我们只需加载示例数据，并使用前文 `rl_zero` 提示语引导语言模型回答问题。

评估指标。在评估多项选择题或二元响应任务时，评估指标非常明确——我们测试模型输出是否完全正确。

在数学问题中，我们假设存在已知的真实值（例如0.5），但无法简单验证模型输出是否精确为0.5——它也可能输出1/2（/答案）。因此，在评估MATH时，我们必须解决从语言模型（LM）获取语义等效响应的棘手问题。

为此，我们设计了一个答案解析函数，该函数以模型输出和已知标准答案作为输入，返回布尔值判断模型输出是否正确。例如，奖励函数可接收模型输出的字符串（如以（答案）She sold 15 clips. (/答案）结尾）和标准答案72，若模型输出正确则返回True，否则返回False（本例中应返回False）。

在我们的MATH实验中，我们将使用一种快速且相当准确的答案解析器，该解析器在近期关于推理强化学习的研究中被使用[Liu等人，2025]。该奖励函数在 `cs336_alignment.drgrpo_grader.r1_zero_reward_fn` 中实现，除非另有说明，否则您应使用它来评估MATH上的性能。

生成超参数。在生成响应时，我们将采样温度为1.0、top-p为1.0、最大生成长度为1024。提示要求模型以字符串 `</answer>` 结束其回答，因此我们可以指示vLLM在模型输出该字符串时停止：

```
# 基于Dr. GRPO：当模型完成回答时停止
# https://github.com/sail-sg/understand-r1-zero/blob/
# c18804602b85da9e88b4aeeb6c43e2f08c594fbc/train_zero_math.py#L167
sampling_params.stop = ["</answer>"]
```

采样参数包括输入字符串 **True**

问题（数学基准）：4分

- (a) 编写脚本以评估Qwen 2.5 Math 1.5B在MATH数据集上的零样本性能。该脚本应(1)从 `/data/a5-alignment/MATH/validation.jsonl` 加载MATH验证示例，
(2) (3)使用 `r1_zero` 提示将它们格式化为语言模型的字符串提示，并为每个示例生成输出。该脚本还应(4)计算评估指标。
(5) 将示例、模型生成结果及相应评估分数序列化并存储至磁盘，以便后续问题分析使用。

在实现过程中，建议添加一个名为 `evaluate_vllm` 的方法，其参数设置可参考以下示例，以便后续调用时能够复用：

```
def evaluate_vllm(
    vllm_model: LLM,
    reward_fn: Callable[[str, str], dict[str, float]],
    prompts: List[str],
    evalsampling_params: SamplingParams) ->
```

无：

```
"""
    Evaluate a language model on a list of prompts,
    计算评估指标，并将结果序列化存储至磁盘。
"""
```

可交付成果：用于评估零样本数学基准性能的脚本。

- (b) 在Qwen 2.5 Math 1.5B上运行评估脚本。模型生成的代数为以下几类：(1) 同时获得格式和答案奖励1的正确生成，(2) 仅获得格式奖励的生成1和回答奖励0，(3) 格式奖励0和回答奖励0？观察到至少10个格式奖励为0的案例，您认为问题出在基础模型的输出上，还是解析器上？为什么？至于（至少10个）格式奖励为1但回答奖励为0的案例呢？**交付成果：**对模型和奖励函数性能的评述，包括各类别的示例。
- (c) Qwen 2.5数学1.5B零样本基线模型在数学测试中的表现如何？

可交付成果1-2句带评估指标的句子。

4 数学的监督微调

算法1 监督微调 (SFT)

输入初始策略模型 $\pi_{\theta_{\text{init}}}$; SFT 数据集 D

- 1: 策略模型 $\pi_{\theta} \leftarrow \pi_{\theta_{\text{init}}}$
- 2: **for** step = 1 , ..., n_sft_steps **do**
- 3: 从 D 中抽取一批问答对 $\mathcal{D}_b$
- 4: 使用模型 π_{θ} 计算给定问题的响应的交叉熵损失
- 5: 通过针对交叉熵损失进行梯度步长更新模型参数 θ
- 6: **end**

输出 π_{θ}

推理能力的监督微调。本节我们将基于MATH数据集对基础模型进行微调（算法1）。由于目标是提升模型的推理能力而非直接预测正确答案，我们采用先生成推理链轨迹再输出答案的微调策略。为此，我们提供了由DeepSeek R1 DeepSeek-AI等获取的推理轨迹数据集。[2025]在

`/data/a5-alignment/MATH/sft.jsonl`

在实际训练推理模型时，通常会先用 SFT 作为第二阶段强化学习（RL）微调的热启动。这主要有两个原因：首先，SFT 需要高质量的标注数据（即带有预先存在的推理轨迹），而RL只需正确答案作为反馈；其次，即使在标注数据充足的情况下，RL仍能通过发现优于 SFT 数据的策略来提升性能。遗憾的是，我们使用的模型规模尚不足以在 SFT 与RL结合时展现效果，因此本任务将这两个阶段分开处理。

4.1 使用HuggingFace模型

加载HuggingFace模型和分词器。若需从本地目录加载HuggingFace模型和分词器（采用bfloat16浮点数格式并使用FlashAttention-2优化内存），可参考以下示例代码：

从transformers导入AutoModelForCausalLM, Auto tokenizer

```
model = AutoModelForCausalLM.from_pretrained(
    "/data/a5-alignment/models/Qwen2.5-Math-1.5B", torch_dtype=
    torch.bfloat16, attn_implementation= "flash_attention_
    2" ,
)
tokenizer = AutoTokenizer.from_pretrained("/data/a5-alignment/models/Qwen2.5-Math-1.5B")
```

正向传递。加载模型后，我们对输入ID批次进行前向传播运算，获取输出的logits（即logits属性）。随后计算模型预测logits与实际标签之间的损失值：

```
input_ids = train_batch["input_ids"].to(device)
labels = train_batch["labels"].to(device)

logits = model(input_ids).logits
loss = F.cross_entropy(..., ...)
```


保存训练完成的模型。要将训练完成后保存模型到指定目录，可调用`.save_pretrained()`函数，并指定输出路径。建议将文件保存至`/data/yourusername`目录，因文件体积较大。同时建议一并保存分词器（即使未修改），这样模型与分词器即可自包含，从单一目录加载。

```
# 保存模型权重
model.save_pretrained(save_directory=output_dir)
tokenizer.save_pretrained(save_directory=output_dir)
```

梯度累积技术。尽管模型采用**bf16**浮点型数据结构并使用FlashAttention-2注意力机制，但即便是80GB显存的GPU也难以支持合理的批量处理规模。为实现更大批量计算，我们可以采用名为**梯度累积**的技术。该技术的核心思想是：无需在每个批次后更新模型权重（即执行优化器步骤），而是将多个批次的梯度值累积后再进行梯度更新。直观来说，若使用更大容量的GPU，我们应当能像一次性处理32个样本批次那样获得相同计算结果。将样本分为16个批次，每批次2个样本，最后取平均值。

梯度累积在PyTorch中实现起来非常简单。需要说明的是，每个权重张量都包含一个存储梯度的属性`.grad`。在调用`loss.backward()`函数前，该属性值为`None`；执行`loss.backward()`后，属性值即为梯度。通常会先执行优化器的一步操作，再通过`optimizer.zero_grad()`将梯度重置为零，从而清空权重张量的`.grad`字段。

对于输入，数据加载器中的标签在：

```
# 正向传递。
logits = model(inputs)
loss = loss_fn(logits, labels)

# 向后传球。
loss.backward()

# 更新权重。
optimizer.step()

# 为下一次迭代做准备，零梯度。
optimizer.zero_grad()
```

为实现梯度累积，我们每隔 k 步调用一次优化器步骤()和优化器零梯度()，其中 k 为梯度累积步数。在调用`loss.backward()`前，我们将损失函数除以梯度累积步数，使梯度在梯度累积步数上取平均值。

```
gradient_accumulation_steps = 4
对于idx, (输入, 标签) 在enumerate(data_loader):
    # 正向传递。
    logits = model(inputs)
    loss = loss_fn(logits, labels) / gradient_accumulation_steps

    # 向后传球。
    loss.backward()

    if (idx + 1) % gradient_accumulation_steps == 0:
        每gradient_accumulation_steps`批次更新一次权重。
        optimizer.step()
```



```
# 每隔 `gradient_accumulation_steps` 个批次执行一次零梯度操作。  
优化器。零梯度()
```

因此，我们在训练时的有效批量大小乘以 k （梯度累积步数）。

4.2 SFT 辅助方法

接下来，我们将实现一些辅助方法，这些方法将在 SFT 和后续强化学习实验中使用。关于术语说明：在后续章节中，我们将交替使用‘输出’、‘补全’或‘响应’来指代模型根据提示生成的补全结果。

提示词与输出的分词处理。对于每个问题与目标输出的配对 (q, o) ，我们将分别对问题和输出进行分词处理并拼接。随后，可使用我们的 SFT 模型（或后续章节中的强化学习策略）对输出的对数概率进行评估。此外，我们需要构建一个响应掩码：该布尔掩码对响应中的所有标记设为**True**，对问题和填充标记设为**False**。在训练循环中，我们将使用该掩码确保仅对响应标记计算损失。

问题（提示词与输出的分词）：提示词与输出的分词（2分）

可交付成果：实现一个名为 `tokenize_prompt_and_output` 的方法，该方法分别对问题和输出进行分词处理，将它们拼接后构建响应掩码。建议采用以下接口：

def tokenize_prompt_and_output(prompt_strs, output_strs, tokenizer):对提示和输出字符串进行分词，并构建一个掩码，该掩码对响应标记为1，对其他标记（提示或填充）为0。

参数：

prompt_strs:`list[str]`提示字符串列表。

output_strs:`list[str]`输出字符串列表。

分词器: `PreTrained tokenizer`用于分词的分词器。

返回：

字典[字符串, torch.张量].设 `prompt_and_output_lens` 为包含分词后的提示和输出字符串长度的列表。则返回的字典应具有以下键：

input_ids torch. 形状为 $(\text{batch_size}, \max(\text{prompt_and_output_lens}) - 1)$ 的张量：

经过分词处理的提示词和输出字符串，最终切分后的分词。

标签火炬。形状为 $(\text{batch_size}, \max(\text{prompt_and_output_lens}) - 1)$ 的张量：移位输入 id，即不包含第一个标记的输入 id。

response_mask torch. 形状为 $(\text{batch_size}, \max(\text{prompt_and_output_lens}) - 1)$ 的张量：用于标记响应标记的掩码。

为测试代码，请实现 `[adapters.run tokenize_prompt_and_output]`。随后使用 `uv run pytest -k test tokenize_prompt_and_output` 执行测试，确保实现方案通过验证。

记录每个标记的熵值。在进行强化学习（RL）时，跟踪每个标记的熵值有助于判断模型的预测分布是否变得过于自信。我们将立即实现这一功能，并比较不同微调方法对模型预测熵的影响。

离散分布 $p(x)$ 的熵，其支持集为 X ，定义为

$$H(p) = - \sum_{x \in X} p(x) \log p(x). \quad (1)$$

根据我们的 SFT 或强化学习模型的logits，我们将计算每个标记的熵，即每个下一个标记预测的熵。

问题（计算熵）：每个标记的熵（1分）

可交付成果：实现 `compute_entropy` 方法，用于计算下一个标记预测的每个标记熵。

建议采用以下接口：

```
def compute_entropy(logits: torch.Tensor) -> torch.Tensor:
```

计算下一个标记预测的熵（即词汇维度上的熵）。参数：

logit: torch.Tensor 形状为 (batch_size, sequence_length, vocab_size) 的张量
包含未标准化的对数几率值。

返回：

火炬。张量形式 (batch_size, sequence_length)。用于每个下一个标记预测的熵值。

注：应使用数值稳定的方法（例如使用 `logsumexp`）以避免溢出。为测试代码，请实现 `[adapters.run_compute_entropy]`。然后运行 `uv run pytest -k test_compute_entropy` 并确保实现通过。

从模型获取对数概率。从模型获取对数概率是我们在 SFT 和强化学习中都需要的基本操作。

对于前缀 x ，一个生成下一个标记的对数概率 $f_{\theta(x)} W R^{|V|}$ 的语言模型，以及一个标签 $y \in V$ ， y 的对数概率为

$$\log p_{\theta}(y | x) = \log [\text{softmax}(f_{\theta(x)})]_y, \quad (2)$$

其中符号 $[x]_y$ 表示向量 x 的第 y 个元素。

您需要采用数值稳定的计算方法来完成此操作，并可自由使用 `torch.nn.functional` 包中的相关方法。我们还建议添加一个参数，用于可选地计算并返回标记熵。

问题（获取响应日志概率）：响应日志概率（及熵值）（2分）

可交付成果：实现 `get_response_log_probs` 方法，该方法可从因果语言模型中获取每个标记的条件对数概率（基于前序标记），并可选地获取模型下一标记分布的熵值。

建议采用以下接口：

```
def get_response_log_probs(
    model: PreTrainedModel,
    input_ids: torch.Tensor,
    labels: torch.Tensor,
    return_token_entropy: bool = False,
) -> dict[str, torch.Tensor]:
```

参数:

模型: 预训练模型用于评分的HuggingFace模型（若需不计算梯度，则置于正确设备上并处于推理模式）。

input_ids: torch.Tensor形状（batch_size, sequence_length），由您的分词方法生成的提示+响应标记的拼接。

标签: torch。张量形状（batch_size, sequence_length），标签由您的分词方法生成。

return_token_entropy: bool If True, also return per-token entropy by calling compute_entropy.

返回:

字典[字符串, torch.张量]。

“log_probs” 形状（batch_size, sequence_length），条件对数概率 $\log p_{\theta}(x_t|x < t)$ 。

“token_entropy” 可选，形状（batch_size, sequence_length），每个位置的单个令牌熵（仅当 return_token_entropy=True时存在）。

实施提示:

- 通过模型（输入ID）获取logits值。

要测试代码，请实现[adapters.run_get_response_log_probs]。随后运行uv run pytest -k test_get_response_log_probs并确保测试通过。

SFT 微批次训练步骤。在 SFT 中我们最小化的损失是给定提示时目标输出的负对数似然。为了计算这个损失，我们需要计算给定提示时目标输出的对数概率，并对输出中的所有标记进行求和，同时屏蔽提示中的标记并填充标记。

我们将为此实现一个辅助函数，该函数在后续强化学习（RL）过程中也将被使用。

问题（masked_normalize）：掩码归一化（1分）

可交付成果: 实现一个名为 maskedNormalize 的方法，该方法对张量元素求和并按常数归一化，同时遵循布尔掩码。

建议采用以下接口:

```
def masked_normalize(
    tensor: torch.Tensor,
    mask: torch.Tensor,
    normalize_constant: float,
    尺寸: 整数|无 = 无,
) -> torch.Tensor:
```

对维度求和并按常数归一化，仅考虑那些掩码==1的元素。

参数：

张量: torch。张量需要求和和归一化的张量。

mask: torch.Tensor与张量形状相同；位置带有1的项被包含在求和中。

normalize_constant: float用于归一化的常数除数。

维度: 整数无正规化前要求和的维度。若为无，则对所有维度求和。

返回：

torch.Tensor归一化后的总和，其中被屏蔽的元素（mask ==0）不参与求和。

为测试代码，请实现[`adapters.run_masked_normalize`]。随后运行`uv run pytest -k test_masked_normalize`并确保其通过。

SFT 微批次训练步骤。现在我们准备为 SFT 实现单个微批次训练步骤（回想一下，对于训练小批次，如果 `gradient_accumulation_steps>1`，我们会迭代多个微批次）。

问题（`sft_microbatch_train_step`）：微批次训练步骤（3分）

交付成果：实现 SFT 的单批次微更新，包括交叉熵损失、掩码求和及梯度缩放。

建议采用以下接口：

```
def sft_microbatch_train_step(策略日志概率:
    torch.Tensor, 响应掩码: torch.Tensor,
    梯度累积步数:int, 归一化常数:float= 1.0,
) -> tuple[torch.Tensor, dict[str, torch.Tensor]]:
```

对微批次执行前向与后向传递。

参数：

policy_log_probs (batch_size, sequence_length)，表示正在训练的 SFT 策略中每个标记的对数概率。

response_mask(batch_size, sequence_length)，1用于响应标记，0用于提示/填充。

gradient_accumulation_steps每优化器步骤的微批次数量。

normalize_constant用于除以总和的常量。默认值为 1.0。返回值：

元组[torch Tensor，字典[字符串， torch Tensor]]。

损失标量张量。微批次损失，已针对梯度累积进行调整。我们返回该值以便进行日志记录。

元数据字典，包含底层损失函数调用的元数据，以及您可能需要记录的其他统计信息。

实施提示：

- 您应在该函数中调用`loss backward()`函数。请确保对梯度累积进行调整。

为测试代码，请实现`[adapters.run_sft_microbatch_train_step]`。随后运行`uv run pytest -k test_sft_microbatch_train_step`并确认其通过。

循环日志记录。在循环中记录模型生成的响应始终是良好实践，推理 SFT /RL也不例外。编写`logGenerations`函数，让模型根据给定提示（例如从验证集采样）生成响应。建议为每个示例至少记录以下内容：

1. 输入提示。
2. SFT /RL模型生成的响应。
3. 标准答案。
4. 奖励信息，包括格式、答案及总奖励。
5. 响应的平均令牌熵。
6. 平均应答时长、正确应答的平均应答时长以及错误应答的平均应答时长。

问题（`log_generations`）：记录世代数（1分）

可交付成果：实现一个名为`logGenerations`的函数，用于记录模型的生成代数。

4.3 SFT 实验

使用上述组件，您现在将实现完整的 SFT 流程（算法1），以在MATH数据集上对Qwen 2.5 Math 1.5B Base模型进行微调。`/data/a5-alignment/MATH/sft.jsonl`中的每个示例包含格式化的提示和目标响应，其中目标响应包含思维链推理轨迹和最终答案。具体而言，每个示例都是类型为`{ "prompt": str, "response": str }`的JSON 元素。

为追踪模型在训练过程中的进展，建议定期在MATH验证集上进行评估。运行脚本时需配置两块GPU，其中一块用于策略模型，另一块用于评估策略的vLLM实例。为实现该功能，以下为初始代码示例，用于初始化vLLM并在每次部署阶段前将策略权重加载至vLLM实例中：

从`vllm.model_executor`导入`set_random_seed`作为`vllm_set_random_seed`

```
def init_vllm(model_id: str, device: str, seed: int, gpu_memory_utilization: float = 0.85):  
    """  
    Start the inference process, here we use vLLM to hold a model on  
    一个与策略分离的GPU。
```

```

"""
vllm_set_random_seed(seed)

# Monkeypatch来自TRL :
# https://github.com/huggingface/trl/blob/
# 22759c820867c8659d00082ba8cf004e963873c1/trl/trainer/grpo_trainer.py
# 更新vLLM补丁以确保我们能够
# (1) place the vLLM model on the desired device (world_size_patch) and
# (2) 避免使用非本研究环境设计的检测方法 (profiling_patch) 。
world_size_patch = patch("torch.distributed.get_world_size", return_value=1)
profiling_patch = patch(
    "vllm_worker worker. Worker._assert_memory_footprint_increased_during_profiling" , return_value=None
)

```

使用 world_size_patch 和 profiling_patch: 返回

```

LLM (
    model=model_id,
    device=device,
    dtype=torch.bfloat16,
    enable_prefix_caching=True,
    gpu_memory_utilization=gpu_memory_utilization,
)

def load_policy_into_vllm_instance(policy: PreTrainedModel, llm: LLM):
    """
    复制自 https://github.com/huggingface/trl/blob/22759c820867c8659d00082ba8cf004e963873c1/trl/trainer/grpo_trainer.py#L670。
    """
    state_dict = policy.state_dict()
    llm_model = llm.llm_engine.model_executor.driver_worker.model_runner.model
    llm_model.load_weights(state_dict.items())

```

建议同时记录训练集和验证集的指标数据（该做法对后续强化学习实验同样具有参考价值）。在wandb平台中，可采用以下代码实现：

```

# 设置wandb指标
wandb.define_metric("train_step") # the x-axis for training
wandb.define_metric("eval_step") # the x-axis for evaluation

# everything that starts with train/ is tied to train_step
wandb.define_metric("train/*", step_metric="train_step")

# everything that starts with eval/ is tied to eval_step
wandb.define_metric("eval/*", step_metric="eval_step") 最后，我

```

们建议您使用梯度剪切，剪切值设为1.0。

问题 (sft_experiment)：在MATH数据集上运行 SFT (2个点) (2 H100小时)

1. 使用Qwen 2.5数学1.0版本对推理 SFT 示例（位于/data/a5-alignment/MATH/sft.jsonl）进行运行 SFT。5B基础模型，通过调整 SFT 中独特示例的数量

范围{128,256,512,1024}，同时使用完整数据集。调整学习率和批量大小，以在使用完整数据集时达到至少15%的验证准确率。

可交付成果：与不同数据集规模相关的验证准确率曲线。

2. 筛选推理 SFT 示例，仅保留能得出正确答案的示例。在（完整）过滤后的数据集上运行 SFT，报告过滤后数据集的规模及验证准确率。

交付成果：报告数据集规模及验证准确率曲线。将结果与先前 SFT 实验进行对比。

5 MATH的专家迭代

在上一节中，我们观察到通过从 SFT 数据中过滤掉不良示例可以提升 SFT 模型的性能。本节我们将更进一步：将此过滤过程应用于由基础模型自身生成的推理轨迹。该过程在文献中被称为专家迭代 [Anthony 等人, 2017]，在语言模型领域已被 Cobbe 等人探索。[2021b]Zelikman 等人[2022]多汉等人[2022]Gulcehre 等人[2023]。

算法2 专家迭代 (EI)

输入初始策略模型 $\dot{U}_{\theta_{\text{init}}}$ ；奖励函数 R ；任务问题 D

- 1: 策略模型 $\dot{U}_{\theta} \leftarrow \dot{U}_{\theta_{\text{init}}}$
- 2: **for** step = 1, ..., n_ei_steps **do**
- 3: 从 D 中抽取一批问题 $\mathcal{D}_b$
- 4: 设置旧策略模型 $\dot{U}_{\theta_{\text{old}}} \leftarrow \dot{U}_{\theta}$
- 5: 样本 G 输出 $\{o^{(i)}\}_{i=1}^G$ 、 $\dot{U}_{\theta_{\text{old}}}(\cdot | q)$ 对于每个问题 $q \in \mathcal{D}_b$
- 6: 计算奖励 $\{r^{(i)}\}_{i=1}^G$ for each sampled output $o^{(i)}$ by running reward function $R(q, o^{(i)})$
- 7: 过滤出错误输出（即， $o^{(i)}$ 与 $r^{(i)} = 0$ ）以获得正确问题-回答对的数据集 $\mathcal{D}_{\text{sft}}$
- 8: $\dot{U}_{\theta} \leftarrow \text{SFT}(\dot{U}_{\theta}, \mathcal{D}_{\text{sft}})$ (算法1)
- 9: **end**

输出 $\dot{U}_{\theta}$

接下来，我们将对 MATH 数据集进行专家迭代。

温馨提示：建议在 vLLM 采样参数中设置 min_tokens 值，这样能有效避免生成空字符串（根据具体实现，空字符串可能导致下游出现 NaN 值）。具体操作方法如下：

```
sampling_min_tokens = 4
sampling_params = SamplingParams(
    temperature=sampling_temperature,
    max_tokens=sampling_max_tokens,
    min_tokens=sampling_min_tokens,
    n=G,
    seed=seed,
)
```

与 SFT 相同，应使用剪切值为 1.0 的渐变剪切功能。

问题 (expert_iteration_experiment)：在 MATH 数据集上运行专家迭代（2分）（6 H100 小时）

在 MATH 数据集（提供于 /data/a5-alignment/MATH/train.jsonl）上执行专家迭代

使用Qwen 2.5数学1.5B基础模型，调整每个问题的 G 轮次数量和 SFT 步骤中使用的迭代次数（ $n_ei_steps=5$ ）。在{512、1024、2048}范围内调整专家迭代步骤的批量大小（即 $\mathcal{D}_b$ 的大小）。（无需尝试所有可能的超参数组合，只需能得出每个参数的结论即可。）记录模型在训练过程中响应的熵值。确保vLLM在第二个答案标签处终止生成，如 SFT 部分所述。

可交付成果：与不同部署配置相关的验证准确率曲线。至少尝试2种不同的部署次数和训练轮次。

可交付成果：在数学测试（MATH）中验证准确率至少达到15%的模型。

交付成果：用两句话简要说明与 SFT 表现的对比，以及在EI各步骤中的表现。

可交付成果：模型响应熵随训练过程变化的曲线图。

6 政策梯度的初探

语言模型研究中一个令人兴奋的新发现是，使用强基础模型对已验证奖励进行强化学习，可以显著提升其推理能力和性能[OpenAI等，2024，DeepSeek-AI等，2025]。最强的此类开放推理模型，如DeepSeek R1和Kimi k1.5[Team等，2025]，是通过策略梯度训练的，这是一种强大的强化学习算法，能够优化任意奖励函数。

我们在下面简要介绍语言模型上的强化学习的策略梯度。我们的介绍基于一些很好的资源，这些资源更深入地介绍了这些概念：OpenAI的《深度强化学习入门》[Achiam，2018a]和Nathan Lambert的《人类反馈的强化学习》（RLHF）[Lambert，2024]。

6.1 语言模型作为政策

一个参数为 θ 的因果语言模型（LM）定义了给定当前文本前缀 s_t （状态/观测）时，下一个标记 a_t 的 WV 的概率分布。在强化学习（RL）的语境中，我们将下一个标记 a_t 视为一个动作，而当前文本前缀 s_t 视为状态。因此，LM是一个分类随机策略

$$a_t \sim \pi_{\theta}(\cdot | s_t), \quad \dot{U}_{\theta}(a_t | s_t) = [\text{softmax}(f_{\theta}(s_t))]_{a_t}. \quad (3)$$

在使用策略梯度优化策略时，需要执行两个基本操作：

1. 从策略中采样：从上述分类分布中抽取一个动作；
2. 对动作的对数似然进行评分：评估对数 $\dot{U}_{\theta}(a_t | s_t)$ 。

一般来说，当使用llm进行强化学习时， s_t 是迄今为止产生的部分完成/解决方案，而每个 a_t 是解决方案的下一个标记；当发出文本结束标记时，如`end_of_text|`，或`</answer>`在我们的 `r1_zero` 提示的情况下，事件结束。

6.2 轨迹

（有限时域）轨迹是指智能体经历的状态与动作的交错序列：

$$\tau = (s_0, a_0, s_1, a_1, \dots, s_T, a_T), \quad (4)$$

其中 T 表示轨迹长度，即 a_T 为文本末尾标记，或达到最大生成标记预算。

初始状态从起始分布中抽取， $s_0 \sim p_0(s_0)$ ；在使用LLMs的强化学习中， $p_0(s_0)$ 是对格式化提示的分布。在一般设置中，状态转移遵循某些环境

动力学 $s_{t+1} \sim P(\cdot | s_t, a_t)$ 。在使用大语言模型的强化学习中，环境是确定性的：下一个状态是旧前缀与发射的标记连接， $s_{t+1} = s_t \cdot a_t$ 。轨迹也称为**剧集**或**卷出**；我们将交替使用这些术语。

6.3 回报与回报

标量奖励 $r_t = R(s_t, a_t)$ 用于评估在状态 s_t 下采取的动作的即时质量。对于已验证域上的强化学习，通常将零奖励分配给中间步骤，并将**已验证奖励**分配给终端动作。

$$r_T = R(s_T, a_T) := \begin{cases} 1 & \text{if the trajectory } s_T \| a_T \text{ matches the ground-truth according to our reward function} \\ 0 & \text{otherwise.} \end{cases}$$

返回 $R(\cdot)$ 沿轨迹聚合奖励。两种常见选择是**有限时域无折扣回报**

$$R(\tau) := \sum_{t=0}^T r_t, \quad (5)$$

和**无限期贴现收益**

$$R(\tau) := \sum_{t=0}^{\infty} \gamma^t r_t, \quad 0 < \gamma < 1. \quad (6)$$

在本研究中，我们将采用未折现的计算公式，因为各事件具有自然终止点（文本末尾或最大生成长度）。

该药物的作用目标是最大化预期回报

$$J(\theta) = \mathbb{E}_{\tau \sim \pi_{\theta}} [R(\tau)], \quad (7)$$

导致优化问题

$$\theta^* = \arg \max_{\theta} J(\theta). \quad (8)$$

6.4 Vanilla政策梯度

接下来，让我们尝试使用**梯度上升法**来学习策略参数 θ ，以期望回报为目标：

$$\theta_{k+1} = \theta_k + \alpha \nabla_{\theta} J(\theta_k). \quad (9)$$

我们将采用的核心身份是强化策略梯度，如下所示。

$$\nabla_{\theta} J(\pi_{\theta}) = \mathbb{E}_{\tau \sim \pi_{\theta}} \left[\sum_{t=0}^T \nabla_{\theta} \log \pi_{\theta}(a_t | s_t) R(\tau) \right]. \quad (10)$$

推导策略梯度。我们是如何得到这个方程的？为完整起见，我们将在下文给出该恒等式的推导过程。我们将运用若干数学恒等式。

1. 轨迹的概率由以下公式给出

$$P(\tau | \theta) = \rho_0(s_0) \prod_{t=0}^T P(s_{t+1} | s_t, a_t) \pi_{\theta}(a_t | s_t). \quad (11)$$

因此，轨迹的对数概率为：

$$\log P(\tau \mid \theta) = \log \rho_0(s_0) + \sum_{t=0}^T [\log P(s_{t+1} \mid s_t, a_t) + \log \pi_\theta(a_t \mid s_t)]. \quad (12)$$

2. 对数导数技巧:

$$\nabla_{\theta} P = P \nabla_{\theta} \log P. \quad (13)$$

3. 环境项在 π_{θ} 中是常数。 ρ_0 、 $P(\cdot|\cdot)$ 和 $R(\cdot)$ 不依赖于策略参数，因此

$$\nabla_{\theta} \rho_0 = \nabla_{\theta} P = \nabla_{\theta} R(\tau) = 0. \quad (14)$$

应用上述事实:

$$\nabla_{\theta} J(\theta) = \nabla_{\theta} \mathbb{E}_{\tau \sim \pi_{\theta}} [R(\tau)] \quad (15)$$

$$= \nabla_{\theta} \sum_{\tau} P(\tau|\theta) R(\tau) \quad (16)$$

$$= \sum_{\tau} \nabla_{\theta} P(\tau|\theta) R(\tau) \quad (17)$$

$$= \sum_{\tau} P(\tau|\theta) \nabla_{\theta} \log P(\tau|\theta) R(\tau) \quad (\text{对数导数技巧}) \quad (18)$$

$$= \mathbb{E}_{\tau \sim \pi_{\theta}} [\nabla_{\theta} \log P(\tau|\theta) R(\tau)], \quad (19)$$

因此，将轨迹的对数概率代入，并利用环境项在 π_{θ} 中保持不变的事实，我们得到 *普通* 或强化策略梯度:

$$\nabla_{\theta} J(\pi_{\theta}) = \mathbb{E}_{\tau \sim \pi_{\theta}} \left[\sum_{t=0}^T \nabla_{\theta} \log \pi_{\theta}(a_t|s_t) R(\tau) \right]. \quad (20)$$

直观而言，该梯度将增加轨迹中每个高回报动作的对数概率，反之则降低其对数概率。

梯度的样本估计。 给定一批 N 个部署 $D = \{\tau^{(i)}\}_{i=1}^N$ 通过采样初始状态 $s_0^{(i)}$ 、 $\rho_0(s_0)$ ，然后在环境中运行策略 π_{θ} ，我们形成一个无偏的梯度估计器

$$\hat{g} = \frac{1}{N} \sum_{i=1}^N \sum_{t=0}^T \nabla_{\theta} \log \pi_{\theta}(a_t^{(i)} | s_t^{(i)}) R(\tau^{(i)}). \quad (21)$$

该向量用于梯度上升更新 $\theta \leftarrow \theta + \hat{g}$.

6.5 策略梯度基线

标准策略梯度的主要问题是梯度估计的高方差。一种常见的缓解方法是从奖励中减去一个仅依赖于状态的 *基线* 函数 b 。这是一种 *控制变量* [Ross, 2022]: 其思路是通过减去一个与估计量相关但不引入偏差的项来降低估计量的方差。

我们将基线策略梯度定义为:

$$B = \mathbb{E}_{\tau \sim \pi_{\theta}} \left[\sum_{t=0}^T \nabla_{\theta} \log \pi_{\theta}(a_t|s_t) (R(\tau) - b(s_t)) \right]. \quad (22)$$

例如，一个合理的基线是策略上值函数 $V^{\pi}(s) = \mathbb{E}_{\tau \sim \pi_{\theta}} [R(\cdot) | s_t = s]$ ，即如果我们从 $s_t = s$ 开始并从那里遵循策

略 $\dot{U}_\theta$ ，那么期望回报。然后，该量 $(R(s_t) - \bar{V}^\pi(s_t))$ 直观上表示实现轨迹比预期轨迹好多少。

只要基线仅取决于状态，基线策略梯度即无偏。我们可以通过将基线策略梯度重写为

$$B = \mathbb{E}_{\tau \sim \pi_\theta} \left[\sum_{t=0}^T \nabla_\theta \log \pi_\theta(a_t | s_t) R(\tau) \right] - \mathbb{E}_{\tau \sim \pi_\theta} \left[\sum_{t=0}^T \nabla_\theta \log \pi_\theta(a_t | s_t) b(s_t) \right]. \quad (23)$$

聚焦于基线项，我们观察到

$$\mathbb{E}_{\tau \sim \pi_\theta} \left[\sum_{t=0}^T \nabla_\theta \log \pi_\theta(a_t | s_t) b(s_t) \right] = \sum_{t=0}^T \mathbb{E}_{s_t} \left[b(s_t) \mathbb{E}_{a_t \sim \pi_\theta(\cdot | s_t)} [\nabla_\theta \log \pi_\theta(a_t | s_t)] \right]. \quad (24)$$

一般来说，得分函数的期望值为零： $\mathbb{E}_{x \sim P_\theta} [\nabla_\theta \log P_\theta(x)] = 0$ 。因此，等式24中的表达式为零且

$$B = \mathbb{E}_{\tau \sim \pi_\theta} \left[\sum_{t=0}^T \nabla_\theta \log \pi_\theta(a_t | s_t) R(\tau) \right] - 0 = \nabla_\theta J(\pi_\theta), \quad (25)$$

因此我们得出结论：基线策略梯度是无偏的。后续我们将通过实验验证基线化是否能提升下游性能。

关于策略梯度“损失”的说明。当我们在PyTorch等框架中实现策略梯度方法时，会定义一个名为`pg_loss`的策略梯度损失函数，调用`pg_loss backward()`后，模型参数的梯度缓冲区将被填充为近似策略梯度。 $\hat{g}$ 在数学中，可以表述为

$$\text{pg_loss} = \frac{1}{N} \sum_{i=1}^N \sum_{t=0}^T \log \pi_\theta(a_t^{(i)} | s_t^{(i)}) (R(\tau^{(i)}) - b(s_t^{(i)})). \quad (26)$$

`pg_loss`并非典型意义上的损失——在训练集或验证集上将其作为评估指标并无实际意义，且良好的验证`pg_loss`并不能表明我们的模型具有良好的泛化能力。`pg_loss`实际上只是一个标量，当我们调用`pg_loss.backward()`时，通过反向传播获得的梯度即为近似策略梯度。 $\hat{g}$ 。

在进行强化学习（RL）时，应始终**记录并报告训练与验证奖励**。这些是“有意义”的评估指标，也是我们试图通过策略梯度方法优化的内容。

6.6 离策略的策略梯度

强化学习是一种**基于策略的**算法：训练数据由我们正在优化的同一策略生成。为直观理解这一点，我们来具体说明强化学习算法：

1. 采样一批部署 $\{s_t^{(i)}\}_{i=1}^N$ 从当前政策 π_θ 。
2. Approximate the policy gradient as $\nabla_\theta J(\pi_\theta) \approx \hat{g} = \frac{1}{N} \sum_{i=1}^N \sum_{t=0}^T \nabla_\theta \log \pi_\theta(a_t^{(i)} | s_t^{(i)}) R(\tau^{(i)})$ 。
3. 使用计算出的梯度更新策略参数： $\theta \leftarrow \theta + \hat{g}$ 。

我们需要进行大量推理以采样新的一批部署，却仅需执行一次梯度步。语言模型（LM）的行为通常无法在单步内发生显著变化，因此这种基于策略的方法效率极低。

非策略梯度策略。在非策略学习中，我们从非当前优化策略的其他策略中采样卷积。流行的策略梯度算法（如PPO和GRPO）的非策略变体使用策略 $\dot{U}_{\theta_{\text{old}}}$ 的先前版本卷积来优化当前策略 $\dot{U}_{\theta}$ 。非策略梯度策略估计是

$$\hat{g}_{\text{off-policy}} = \frac{1}{N} \sum_{i=1}^N \sum_{t=0}^T \frac{\pi_{\theta}(a_t^{(i)} | s_t^{(i)})}{\pi_{\theta_{\text{old}}}(a_t^{(i)} | s_t^{(i)})} \nabla_{\theta} \log \pi_{\theta}(a_t^{(i)} | s_t^{(i)}) R(\tau^{(i)}). \quad (27)$$

This looks like an importance sampled version of the vanilla policy gradient, with reweighting terms $\frac{\pi_{\theta}(a_t^{(i)} | s_t^{(i)})}{\pi_{\theta_{\text{old}}}(a_t^{(i)} | s_t^{(i)})}$.

确实，等式27可以通过重要性抽样推导得出，并应用一个合理的近似，只要 $\dot{U}_{\theta}$ 和 $\dot{U}_{\theta_{\text{old}}}$ 差异不大：参见Degris等人[2013] 更多详情请参阅。

7 群组相对策略优化

接下来我们将介绍群组相对策略优化（GRPO），这是您将用于解决数学问题的策略梯度变体。

7.1 GRPO 算法

优势估计。GRPO 的核心思想是从策略 $\dot{U}_{\theta}$ 中为每个问题采样多个输出，并利用它们来计算基线。这很便利，因为我们无需学习神经值函数 $V_{\phi}(s)$ ，该函数可能难以训练且从系统角度来看操作繁琐。对于问题 q 和组输出 $\{o^{(i)}\}_{i=1}^G$ 、 $\dot{U}_{\theta}(\cdot|q)$ ，令 $r^{(i)}=R(q, o^{(i)})$ 为第 i 个输出的奖励。DeepSeekMath [Shao 等人，2024] 和 DeepSeek R1 [DeepSeek-AI 等人，2025] 计算第 i 个输出的群归一化奖励为

$$A^{(i)} = \frac{r^{(i)} - \text{mean}(r^{(1)}, r^{(2)}, \dots, r^{(G)})}{\text{std}(r^{(1)}, r^{(2)}, \dots, r^{(G)}) + \text{advantage_eps}}, \quad (28)$$

其中 `advantage_eps` 是一个防止除以零的小常数。注意这个优势 $A^{(i)}$ 对响应中的每个标记都相同，即 $A_t^{(i)}=A^{(i)}$ ， $\mathcal{E}t W 1, \dots, |o^{(i)}|$ ，因此我们在下文中省略 t 下标。

高级算法。在深入探讨 GRPO 目标之前，我们先通过编写Shao等人提出的算法来了解列车循环的概念。[2024] 在算法3.²中

GRPO 目标。GRPO 目标融合了三个核心理念：

1. 离策略的策略梯度，如等式27所述。
2. 计算优势 $A^{(i)}$ 与群归一化，如等式28所示。
3. 一种剪枝机制，如近端策略优化（PPO，Schulman等人[2017]）。

剪枝的目的是在单批次展开时保持稳定性，当进行大量梯度步时。它通过保持策略 $\dot{U}_{\theta}$ 不会偏离旧策略太远来实现。

²这是DeepSeekMath GRPO 的一个特例，其奖励函数经过验证，不包含KL项，且不迭代更新参考模型和奖励模型。

算法3组相对策略优化 (GRPO)

输入初始策略模型 $\dot{U}_{\theta_{\text{init}}}$; 奖励函数 R ; 任务问题 D

```

1: 策略模型  $\dot{U}_{\theta} \leftarrow \dot{U}_{\theta_{\text{init}}}$ 
2: for step = 1 , ..., n_grpo_steps do
3: 从  $D$  中抽取一批问题  $\mathcal{D}_b$ 
4: 设置旧策略模型  $\dot{U}_{\theta_{\text{old}}} \leftarrow \dot{U}_{\theta}$ 
5: 样本  $G$  输出  $\{o^{(i)}\}_{i=1}^G$ 、  $\dot{U}_{\theta_{\text{old}}}(\cdot | q)$  对于每个问题  $q \in \mathcal{D}_b$ 
6: 计算奖励  $\{r^{(i)}\}_{i=1}^G$  for each sampled output  $o^{(i)}$  by running reward function  $R(q, o^{(i)})$ 
7: 使用群归一化计算  $A^{(i)}$  ( $< \text{term\_0} > 28$ )
8: for train step = 1 , ..., n_train_steps_per_rollout_batch do
9:     通过最大化 GRPO -Clip 目标 (待讨论, 等式29) 来更新策略模型  $\dot{U}_{\theta}$ 
10: 端
11: 端
输出  $\dot{U}_{\theta}$ 

```

我们先列出完整的 GRPO -Clip 目标, 这样就能直观理解剪切操作的作用:

$$J_{\text{GRPO-Clip}}(\theta) = \mathbb{E}_{q \sim D, \{o^{(i)}\}_{i=1}^G \sim \pi_{\theta}(\cdot | q)}$$

$$\left[\frac{1}{G} \sum_{i=1}^G \frac{1}{|o^{(i)}|} \sum_{t=1}^{|o^{(i)}|} \min \left(\frac{\pi_{\theta}(o_t^{(i)} | q, o_{<t}^{(i)})}{\pi_{\theta_{\text{old}}}(o_t^{(i)} | q, o_{<t}^{(i)})} A^{(i)}, \underbrace{\text{clip} \left(\frac{\pi_{\theta}(o_t^{(i)} | q, o_{<t}^{(i)})}{\pi_{\theta_{\text{old}}}(o_t^{(i)} | q, o_{<t}^{(i)})}, 1 - \epsilon, 1 + \epsilon \right)}_{\text{每令牌目标}} A^{(i)} \right) \right].$$

每令牌目标

(29)

超参数 $\epsilon > 0$ 控制策略的可变范围。为直观理解, 我们可以参考 Achiam[2018a, b], 将每个标记的目标函数改写为更直观的形式。定义该函数

$$g(\epsilon, A^{(i)}) = \begin{cases} (1 + \epsilon) A^{(i)} & \text{if } A^{(i)} \geq 0 \\ (1 - \epsilon) A^{(i)} & \text{if } A^{(i)} < 0. \end{cases} \quad (30)$$

我们可以将每个标记的目标重写为

$$\text{per-token objective} = \min \left(\frac{\pi_{\theta}(o_t^{(i)} | q, o_{<t}^{(i)})}{\pi_{\theta_{\text{old}}}(o_t^{(i)} | q, o_{<t}^{(i)})} A^{(i)}, g(\epsilon, A^{(i)}) \right)$$

我们现在可以按案例进行推理。当优势 $A^{(i)}$ 为正时, 每个令牌的目标就简化为

$$\text{per-token objective} = \min \left(\frac{\pi_{\theta}(o_t^{(i)} | q, o_{<t}^{(i)})}{\pi_{\theta_{\text{old}}}(o_t^{(i)} | q, o_{<t}^{(i)})}, 1 + \epsilon \right) A^{(i)}.$$

由于 $A^{(i)} > 0$, 当动作 $o_t^{(i)}$ 在 $\dot{U}_{\theta}$ 下变得更可能时, 目标值上升, 即, 如果 $\dot{U}_{\theta}(o_t^{(i)} | q, o_{<t}^{(i)})$ 增加。最小限制的裁剪决定了目标函数可以增加的幅度: 一旦 $\dot{U}_{\theta}(o_t^{(i)} | q, o_{<t}^{(i)}) > (1 + \epsilon) \dot{U}_{\theta_{\text{old}}}(o_t^{(i)} | q, o_{<t}^{(i)})$, 这个每令牌目标函数达到其最大值 $(1 + \epsilon) A^{(i)}$ 。因此, 策略 $\dot{U}_{\theta}$ 不会被激励偏离旧策略 $\dot{U}_{\theta_{\text{old}}}$ 。

类似地, 当优势 $A^{(i)}$ 为负时, 模型试图降低 $\dot{U}_{\theta}(o_t^{(i)} | q, o_{<t}^{(i)})$, 但没有动力将其降至 $(1 - \epsilon) \dot{U}_{\theta_{\text{old}}}(o_t^{(i)} | q, o_{<t}^{(i)})$ 。

$<^{(i)}_r$ 以下（完整论证参见Achiam[2018b]）。

7.2 实施

在我们对 GRPO 训练循环及其目标有了高层次理解后，将开始实施其具体部分。SFT 和EI章节中实施的许多部分也将被 GRPO 重复使用。

计算优势（群组归一化奖励）。首先，我们将实现计算每个示例在展开批次中的优势逻辑，即群组归一化奖励。我们将考虑两种可能的获取群组归一化奖励的方法：上述等式28中提出的方法，以及一种最近的简化方法。

GRPO 博士[刘等人, 2025]指出，通过 $\text{std}(r^{(1)}, r^{(2)}, \dots, r^{(G)})$ 进行归一化会奖励答案正确性变化较小的批量问题，这可能并不理想。他们建议直接移除归一化步骤，计算

$$A(i) = r(i) - \text{mean}(r(1), r(2), \dots, r(G)). \quad (31)$$

我们将实施两种变体方案，并在后续任务中比较其性能表现。

问题（计算组标准化奖励）：组标准化（2分）

交付成果：实现一个名为`compute_group_normalized_rewards`的方法，该方法计算每个部署响应的原始奖励，按组进行归一化处理，并返回归一化和原始奖励，以及任何你认为有用的元数据。

建议采用以下接口：

```
def 计算归一化奖励组 (
    reward_fn,
    rollout_responses,
    repeated_ground_truths,
    group_size,
    advantage_eps,
    标准化标准:
```

计算每组推广响应的奖励值，并按组别规模进行标准化处理。

参数：

reward_fn: Callable[[**str**, **str**], **dict**[**str**, **float**]] 将策略响应与真实值进行评分，生成包含键“reward”、“format_reward”和“answer_reward”的字典。

rollout_responses: list[**str**] 来自策略的部署。此列表的长度为 `rollout_batch_size = n_prompts_per_rollout_batch * group_size`。

重复的真实情况：列表[字符串] 示例的基准数据。该列表长度为 `rollout_batch_size`，因为每个示例的基准数据会重复 `group_size` 次。

group_size: int 每个问题（组）的响应数量。

advantage_eps: float 用于归一化时避免除零的小常量。

normalize_by_std: bool 若 **True**，则除以每组标准差；否则仅减去组均值。

返回：

元组[torch.Tensor, torch.Tensor, 字典[字符串, 浮点数]]。

优势形状（`rollout_batch_size`）。每个rollout响应的组归一化奖励。

raw_rewards形状 (rollout_batch_size)。每个rollout响应的未标准化奖励。

元数据您可选择记录其他统计数据（例如奖励的均值、标准差、最大值/最小值）。

为测试代码，请实现[`adapters.run_compute_group_normalized_rewards`]。随后，使用`uv run pytest -k test_compute_group_normalized_rewards`执行测试，并确保实现通过。

基于朴素策略梯度的损失函数。接下来，我们将实现若干用于计算“损失”的方法。

作为提醒/免责声明，这些并非真正意义上的损失，不应作为评估指标报告。在强化学习（RL）中，您应跟踪训练和验证回报等指标（参见第6.5节讨论）。

我们将从朴素的策略梯度损失开始，它只是将优势乘以动作的对数概率（并取反）。对于问题 q 、响应 o 和响应标记 o_t ，朴素的逐标记策略梯度损失是

$$-A_t \cdot \log p_{\theta}(o_t|q, o <_t). \quad (32)$$

问题 (compute_naive_policy_gradient_loss)：朴素策略梯度（1分）

可交付成果：实现名为 `compute_naive_policy_gradient_loss` 的方法，该方法使用原始奖励或预计算优势来计算每个标记的策略梯度损失。

建议采用以下接口：

```
def compute_naive_policy_gradient_loss(原始奖励或优势: torch.Tensor, 策略日志概率: torch.Tensor,
) -> torch.Tensor:
```

在每个标记处计算策略梯度损失，其中`raw_rewards_or_advantages`表示原始奖励或已标准化的优势值。

参数：

原始奖励或优势：`torch.张量`形状 (batch_size, 1)，每个回放响应的标量奖励/优势。

policy_log_probs：`torch.Tensor` Shape (batch_size, sequence_length)，每个标记的logprobs。

返回：

`torch.Tensor` Shape (batch_size, sequence_length)，每个标记的策略梯度损失（将在训练循环中跨批次和序列维度聚合）。

实施提示：

- 将原始奖励或优势沿序列长度维度进行广播。为测试代码，请实现[`adapters.run_compute_naive_policy_gradient_loss`]。随后运行`uv run pytest -k test_compute_naive_policy_gradient_loss`并确保测试通过。

GRPO -Clip损失函数。接下来，我们将实现更有趣的 GRPO -Clip损失函数。

每个标记的 GRPO -Clip损失函数

$$-\min \left(\frac{\pi_{\theta}(o_t|q, o_{<t})}{\pi_{\theta_{\text{old}}}(o_t|q, o_{<t})} A_t, \text{clip} \left(\frac{\pi_{\theta}(o_t|q, o_{<t})}{\pi_{\theta_{\text{old}}}(o_t|q, o_{<t})}, 1 - \epsilon, 1 + \epsilon \right) A_t \right). \quad (33)$$

问题 (compute_grpo_clip_loss) : GRPO -Clip损失 (2分)

可交付成果: 实现计算单个标记 GRPO -Clip损失的计算方法 compute_grpo_clip_loss。

建议采用以下接口:

```
def compute_grpo_clip_loss( 优点: torch.
    Tensor, policy_log_probs: torch. Te-
    nsor, old_log_probs: torch. Tensor,
    cliprange:float,
) -> tuple[torch.Tensor, dict[str, torch.Tensor]]:
```

参数:

优势: 火炬。张量形式 (batch_size, 1), 每个示例的优势 A 。

policy_log_probs: torch.Tensor Shape (batch_size, sequence_length), 表示正在训练的策略的每个标记的对数概率。

old_log_probs: torch.Tensor Shape (batch_size, sequence_length), 旧策略的每个标记对数概率。

cliprange:float Clip 参数 ϵ (例如 0.2)。

返回:

元组[torch Tensor, 字典[字符串, torch Tensor]]。

损失火炬。形状为 (batch_size, sequence_length) 的张量, 表示每个标记的截断损失。

元数据字典, 可记录任何需要记录的信息。建议记录每个标记是否被截断, 即最小值右侧的截断策略梯度损失是否低于 LHS。

实施提示:

- 广播优势优于序列长度。

为测试代码, 请实现 [adapters.run_compute_grpo_clip_loss]。随后运行 `uv run pytest -k test_compute_grpo_clip_loss` 并确保测试通过。

策略梯度损失包装器。我们将通过消融实验对比三种不同版本的策略梯度:

(a) 无基线: 未使用基线的朴素策略梯度损失, 即优势仅为原始奖励 $A=R(q, o)$ 。

(b) reinforce_with_baseline: 朴素的策略梯度损失, 但使用我们的组归一化奖励作为优势。如果 r 是来自 compute_group_normalized_rewards 的组归一化奖励 (可能或可能未通过组标准差归一化), 则 $A=r$ 。

(c) `grpo_clip`: GRPO -Clip损失函数。

为便于操作，我们将实现一个封装器，使我们能够轻松在这三种策略梯度损失之间进行切换。

问题（计算策略梯度损失）：策略梯度包装器（1分）

可交付成果：实现 `compute_policy_gradient_loss`，这是一个便捷的包装器，可调度至正确的损失函数（`no_baseline`、`reinforce_with_baseline` 或 `grpo_clip`），并返回每个标记的损失值及任何辅助统计信息。

建议采用以下接口：

```
def compute_policy_gradient_loss(
    policy_log_probs: torch.Tensor, loss_type: Literal[ "no_baseline" , "reinforce_with_baseline" , "grpo_clip" ], raw_rewards: torch.Tensor | None=None,
    优点: torch.Tensor | None=None,
    old_log_probs: torch.Tensor | None=None,
    测距范围: 浮点 | None=None, )->元组[torch.Tensor, 字典[字符串, torch.Tensor]]:
```

选择并计算所需的策略梯度损失。

参数：

policy_log_probs（`batch_size`, `sequence_length`），表示正在训练的策略中每个标记的对数概率。

loss_type为 `"no_baseline"`、`"reinforce_with_baseline"` 或 `"grpo_clip"` 之一。

raw_rewards若 `loss_type == "no_baseline"` 则必需；形状（`batch_size`, 1）。

优势需要用于 `"reinforce_with_baseline"` 和 `"grpo_clip"`；形状（`batch_size`, 1）。

old_log_probs为 `"grpo_clip"` 所需；形状（`batch_size`, `sequence_length`）。

cliprange为 `"grpo_clip"` 所需；用于裁剪的标量'。

返回：

元组[`torch.Tensor`, **字典**[`字符串`, `torch.Tensor`]]。

损失（`batch_size`, `sequence_length`），按词元损失。

元数据字典，来自底层例程的统计信息（例如 GRPO -Clip的剪辑比例）。实现提示：

- 委托计算朴素策略梯度损失或计算组剪切损失。
- 执行参数校验（参见上文断言模式）。
- 将所有返回的元数据整合至单一字典中。

为测试代码，请实现 `[adapters.run_compute_policy_gradient_loss]`。随后运行 `uv run pytest -k test_compute_policy_gradient_loss`并验证其通过。

假设均值。此前我们已掌握计算优势值、对数概率、分词损失以及诸如每个分词的熵值和剪切比例等有用统计量所需的逻辑。为将形状为（`batch_size`, `sequence_length`）的每个分词损失张量转换为损失向量（每个样本对应一个标量），我们

将计算序列维度上的损失均值，但仅针对响应对应的索引（即掩码[i, j] == 1的标记位置）。

在大多数代码库中，通过序列长度进行归一化是使用大语言模型（LLMs）进行强化学习（RL）的标准做法，但显然这并非最佳选择——通过查看我们在文献（21）中对策略梯度估计的阐述，您可能会注意到这一点。), that there is no normalization factor $\frac{1}{T(i)}$ 我们将从这个标准原语开始，通常称为masked_mean，但后续将使用 SFT 期间实现的masked_normalize方法进行测试。

我们将允许指定计算均值的维度，如果dim为None，我们将计算所有掩码元素的均值。这对于获取响应令牌、剪切分数等的每个令牌平均熵可能很有用。

问题（masked_mean）：掩蔽均值（1分）

可交付成果：实现一个名为 masked_mean 的方法，该方法在尊重布尔掩码的前提下对张量元素进行平均计算。

推荐使用以下接口：def masked_mean(

```
    tensor: torch.Tensor,  
    mask: torch.Tensor,  
    尺寸: 整数|无 = 无,  
) -> torch.Tensor:
```

计算张量沿给定维度的均值，仅考虑那些mask == 1的元素。

参数：

张量: torch。张量需要平均的数据。

mask: torch.Tensor与张量形状相同；位置带有1的会被包含在均值中。

维度: 整数|无需要计算平均值的维度。若无，则对所有被屏蔽的元素计算平均值。

返回：

torch。Tensor被掩码的均值；形状与 tensor.mean(dim) 的语义匹配。

为测试代码，请实现[adapters.run_masked_mean]。随后运行uv run pytest -k test_masked_mean并确保其通过。

GRPO 微批次训练步骤。现在我们准备为 GRPO 实现单个微批次训练步骤（回想一下，对于训练小批次，如果gradient_accumulation_steps>1，我们会迭代多个微批次）。

具体而言，基于原始奖励或优势及对数概率，我们将计算每个标记的损失，使用masked_mean将损失聚合为每个样本的标量损失，对批次维度进行平均，调整梯度累积，并进行反向传播。

问题（grpo_microbatch_train_step）：微批次训练步骤（3分）

交付成果：实现 GRPO 的单批次微更新，包括策略梯度损失、掩码平均化和梯度缩放。

建议采用以下接口：

```
def grpo_microbatch_train_step( 策略日志概
    率: torch.Tensor, 响应掩码: torch.
    Tensor, 梯度累积步数:int,
    loss_type: Literal[ "no_baseline" , "reinforce_with_baseline" , "grpo_clip" ], raw_
    rewards: torch.Tensor|None=None,
    优点: torch.Tensor|无=无,
    old_log_probs: torch.Tensor|None=None,
    测距范围: 浮点|无=无, )->元组[torch.Tensor, 字典[字
    符串, torch.Tensor]]:
```

对微批次执行前向与后向传递。

参数：

policy_log_probs (batch_size, sequence_length)，表示正在训练的策略中每个标记的对数概率。

response_mask(batch_size, sequence_length)，1用于响应标记，0用于提示/填充。

梯度累积步骤每个优化器步骤的微批次数量。

loss_type为 “no_baseline” 、 “reinforce_with_baseline” 、 “grpo_clip” 之一。

raw_rewards当 loss_type == “no_baseline” 时需要；形状 (batch_size, 1)。

优势当 loss_type 不等于 “no_baseline” 时需要；形状 (batch_size, 1)。

old_log_probs GRPO -Clip所需参数；形状 (batch_size, sequence_length)。

cliprange Clip参数’ 用于 GRPO -Clip。

返回：

元组[torch Tensor, 字典[字符串, torch Tensor]]。

损失标量张量。微批次损失，已针对梯度累积进行调整。我们返回该值以便进行日志记录。

元数据字典，包含底层损失函数调用的元数据，以及您可能需要记录的其他统计信息。

实施提示：

- 您应在该函数中调用loss backward()函数。请确保对梯度累积进行调整。

为测试代码，请实现[`adapters.run_grpo_microbatch_train_step`]。随后运行 `uv run pytest -k test_grpo_microbatch_train_step`并确认其通过。

综合起来：GRPO 的训练循环。现在我们将为 GRPO 构建一个完整的训练循环。请参考第7.1节的算法来了解整体结构，并在适当情况下采用我们已实现的方法。

下文提供若干初始超参数。若实现正确，使用这些参数应可获得合理结果。

```
n_grpo_steps: int = 200
learning_rate: float = 1e-5
```



```

advantage_eps: float = 1e-6
rollout_batch_size: int = 256
group_size: int = 8
sampling_temperature: float = 1.0
sampling_min_tokens: int = 4 # As in Expiter, disallow empty string responses
sampling_max_tokens: int = 1024
epochs_per_rollout_batch: int = 1 # On-policy
train_batch_size: int = 256 # On-policy
gradient_accumulation_steps: int = 128 # microbatch size is 2, will fit on H100
gpu_memory_utilization: float = 0.85
loss_type: Literal[
    "no_baseline",
    "reinforce_with_baseline",
    grpo_clip, ] = "reinforce_
with_baseline"
use_std_normalization: bool = True
optimizer = torch.optim.AdamW (
    policy.parameters(), lr=learni-
    ng_rate, weight_decay=0.0,
    betas=(0.9, 0.95),
)

```

这些默认超参数将使你处于策略开启设置中——对于每个rollout批次，我们只进行一次梯度步。就超参数而言，这意味着train_batch_size等于rollout_batch_size，而epochs_per_rollout_batch等于1。

以下是一些用于验证逻辑正确性的断言和常量，可排除边缘情况并引导您走向正确的方向：

```

assert train_batch_size % gradient_accumulation_steps == 0, (
    train_batch_size必须能被gradient_accumulation_steps整除
micro_train_batch_size = train_batch_size // gradient_accumulation_steps assert rollout_
batch_size % group_size == 0, (
    部署批次大小必须能被分组大小整除
n_prompts_per_rollout_batch = rollout_batch_size // group_size assert train_
batch_size >= group_size, (
    "train_batch_size must be greater than or equal to group_size"
)
n_microbatches_per_rollout_batch = rollout_batch_size // micro_train_batch_size

```

此外，还有以下几点建议：

- 请记得使用 `r1_zero` 提示，并指示vLLM在第二个答案标签处停止生成，如先前实验所示。
- 我们建议使用typer进行参数解析。
- 采用梯度裁剪，裁剪值设为1.0。
- 应定期记录验证奖励（例如每5或10步记录一次）。为比较超参数，需至少评估1024个验证样本，因CoT/RL评估可能存在噪声。
- 在我们对损失函数的实现中，GRPO-Clip仅应在非策略状态下使用（因其需要旧的对数概率）。
- 在每轮迭代批次包含多个梯度更新周期的非策略设置中，重新计算每个周期的旧对数概率将造成资源浪费。因此，我们可以直接计算旧对数概率。

每次训练时仅使用一次并重复使用。

- 不应根据旧的对数概率进行区分。
- 每次优化器更新时，您需要记录以下部分或全部内容：
 - 损失。
 - 梯度范数
 - 令牌熵
 - 若不符合政策要求，则采用片段分段法。
 - 训练奖励（总量、格式及答案）。
 - 其他任何你认为可能对调试有帮助的内容。

问题（`grpo_train_loop`）：GRPO 训练循环（5分）

交付成果：为 GRPO 实现完整的训练循环。开始在 MATH 上训练策略，并确认验证奖励持续提升，同时确保逐步合理地实施。提供验证奖励与步骤的关联图表，并展示若干随时间变化的示例实施过程。

8 GRPO 实验

现在我们可以开始对 GRPO 训练循环进行实验，尝试不同的超参数和算法微调。每个实验将使用 2 个 GPU，一个用于 vLLM 实例，另一个用于策略。

关于提前终止运行的说明。若在 200 GRPO 步骤前发现超参数间存在显著差异（例如若配置偏离目标或明显次优，您当然可以随时提前终止实验，从而为后续运行节省时间和计算资源。下文所述的 GPU 运行时长仅为粗略估算。

问题（`grpo_learning_rate`）：调整学习率（2分）（6 H100 小时）

基于上述建议的超参数，对学习率进行扫描，并报告最终验证答案的奖励（若优化器发散则记录发散情况）。

可交付成果：与多种学习率相关的验证奖励曲线。

可交付成果：在数学测试（MATH）中验证准确率至少达到 25% 的模型。

可交付成果：用两句话简要说明你在其他记录指标中发现的任何其他趋势。

在后续实验中，可采用上述筛选过程中表现最优的学习率。

基线效应分析。在保持原有超参数（仅调整学习率）的基础上，我们将重点研究基线效应的影响。由于当前采用的是策略上行设置，因此需要对比不同损失函数的表现：

- 无基线
- 加强基准线

注意使用标准正态化（`use_std_normalization`）是默认超参数中为 **True**。

问题（基线组）：基线效应（2分）（2个 H100 小时）

使用 `reinforce_with_baseline` 和 `noBaseline` 训练策略。

可交付成果：与每种损失类型相关的验证奖励曲线。

可交付成果：用两句话简要说明你在其他记录指标中观察到的任何其他趋势。

在接下来的若干实验中，应采用上述实验中确定的最佳损失函数类型。

长度归一化。正如我们在实现`masked_mean`时所暗示的，对损失函数进行序列长度平均既非必要，也不正确。如何求和损失函数的选择是一个关键超参数，它会导致策略动作信用归因的类型不同。

让我们通过兰伯特[2024]中的一个例子来说明这一点。在检查 GRPO 训练步骤时，我们首先获取每个标记的策略梯度损失（暂时忽略剪裁操作）：

```
advantages # (batch_size, 1)
per_token_probability_ratios # (batch_size, sequence_length)
per_token_loss = -advantages * per_token_probability_ratios
```

我们已阐述了序列长度的优势。现将两种聚合这些单个标记损失的方法进行比较：

- 我们实现的`masked_mean`方法，是对每个序列中未掩码标记进行平均处理。
- 对每个序列中的未掩码标记求和，并除以一个常数标量（我们的`masked_normalize`方法支持常数`normalizer != 1.0`）[Liu等人，2025，Yu等人，2025]。

我们将考虑一个示例，其中批量大小为2，第一个响应包含4个标记，第二个响应包含7个标记。随后，我们可以观察这些归一化方法如何影响梯度。

从`your_utils`导入`masked_mean`，`masked_normalize`

```
ratio = torch.tensor([
    [1, 1, 1, 1, 1, 1, 1,],
    [1, 1, 1, 1, 1, 1, 1,],
], requires_grad=True)

advs = torch.tensor([
    [2, 2, 2, 2, 2, 2, 2,],
    [2, 2, 2, 2, 2, 2, 2,],
])

masks = torch.tensor([
    # 第一代: 4个标记
    [1, 1, 1, 1, 0, 0, 0,],
    # 第二代: 7个标记
    [1, 1, 1, 1, 1, 1, 1,],
])

# 采用每种方法进行标准化
max_gen_len = 7
masked_mean_result = masked_mean(ratio * advs, masks, dim=1)
masked_normalize_result = masked_normalize(
    ratio * advs, masks, dim=1, constant_normalizer=max_gen_len)

print("masked_mean", masked_mean_result)
print("masked_normalize", masked_normalize_result)

# masked_mean tensor([2., 2.], grad_fn=<DivBackward0>)
# masked_normalize tensor([1.1429, 2.0000], grad_fn=<DivBackward0>)

masked_mean_result.mean().backward()
```

```
print("ratio.grad", ratio.grad)
# 比例。
```

```
# 张量[[0.2500,0.2500,0.2500,0.2500,0.0000,0.0000,0.0000], #
[0.1429,0.1429,0.1429,0.1429,0.1429,0.1429,0.1429]]比率.梯度.零_()

masked_normalize_result.mean().backward()
print("ratio.grad", ratio.grad)
# 比例。
# 张量 [[0.1429,0.1429,0.1429,0.1429,0.0000,0.0000,0.0000], #      [0.1429,
0.1429, 0.1429, 0.1429, 0.1429, 0.1429]])
```

问题（思考长度归一化）：思考长度归一化（1分）

交付成果：比较两种方法（尚未进行实验）。每种方法的优缺点是什么？是否存在特定场景或示例使某方法表现更优？

现在，让我们通过实验比较masked_mean与masked_normalize。

问题（grpo_length_normalization）：长度归一化效应（2分）（2 H100小时）

交付成果：通过端到端 GRPO 训练运行，对比masked_mean与masked_normalize的归一化效果。报告验证答案奖励曲线。对研究结果进行评述，包括任何呈现显著趋势的其他指标。

提示：考虑与稳定性相关的指标，例如梯度范数。

在后续实验中采用性能更优的长度标准化方法进行修正。

采用组标准差进行归一化。回顾我们对 compute_group_normalized_rewards 的标准实现（基于Shao 等人[2024]DeepSeek-AI等人[2025]），其中我们通过组标准差进行标准化。Liu等人[2025] 研究指出，若采用组标准差进行除法运算，可能对训练过程产生不良影响：标准差较低的问题（例如难度过低或过高的题目，其所有奖励值几乎均为1或0）在训练过程中会被赋予更高的权重。

刘等[2025] 建议移除通过标准差进行归一化的处理，该操作我们已在compute_group_normalized_rewards中实现，现将进行测试。

问题（grpo_group_standard_deviation）：标准差归一化效应（2分）（2 H100小时）

可交付成果：比较使用标准正态化（use_std_normalization == **True**）与使用标准正态化（use_std_normalization == **False**）的性能表现。报告验证答案奖励曲线。对研究结果的评述，包括任何具有显著趋势的其他指标。

提示：考虑与稳定性相关的指标，例如梯度范数。

针对以下实验，采用性能更优的组归一化方法进行修正。

离线策略与在线策略的对比。我们目前实验的超参数均为在线策略：每个运行批次仅执行单次梯度更新，因此我们几乎完全采用了“原则性”策略。

近似 $\hat{g}$ 除上述提及的长度和优势归一化选择外，还涉及策略梯度。

虽然这种方法在理论上是合理的且稳定的，但效率低下。每次部署都需要从策略中缓慢生成，因此成为 GRPO 的主要成本；每次部署仅执行单个梯度步似乎效率低下，这可能不足以显著改变策略的行为。

我们将进行离策略训练实验，即在每个运行批次中执行多次梯度步（甚至多次训练周期）。

问题（grpo_off_policy）：策略 GRPO 的实现

可交付成果：政策 GRPO 培训的实施。

根据您对上述完整 GRPO 循环的实现方式，可能已具备相关基础设施。若尚未具备，则需实施以下内容：

- 每个部署批次应支持多次梯度步长迭代，其中迭代次数和优化器更新次数由部署批次大小、每次部署批次的迭代次数 v 以及训练批次大小控制。
- 修改主训练循环，以便在每次生成批次后、梯度步内循环前从策略获取响应日志概率（即旧日志概率）。建议使用`torch.inference_mode()`函数。
- 您应该使用 “GRPO 剪辑” 损失类型。

现在我们可以通过每轮迭代批次的训练周期数和优化器更新次数来控制偏离策略的程度。

问题（grpo_off_policy_sweep）：离策略 GRPO 超参数扫描（4个点）（12 H100小时）

交付成果：固定`rollout_batch_size = 256`，选择`epochs_per_rollout_batch`和`train_batch_size`的范围进行扫描。首先在有限数量的 GRPO 步骤（<50）上进行广泛扫描以了解性能情况，然后在更大数量的 GRPO 步骤（200）上进行更集中的扫描。提供简短的实验日志以解释所选范围。

与您的策略运行（`epoch_per_rollout_batch = 1`且`train_batch_size = 256`）进行对比，报告关于验证步骤数量及实际运行时间的图表。

报告验证答案奖励曲线。对研究结果进行评述，包括任何具有显著趋势的其他指标（如熵值和响应长度）。将模型在训练期间的响应熵值与你在EI实验中观察到的结果进行比较。

提示：为保持内存使用量恒定，需调整`gradient_accumulation_steps`参数。

在非策略设置中消融剪裁。回想 GRPO -Clip中剪裁的作用是防止策略在单次迭代批次中通过多次梯度更新偏离原有策略。接下来，我们将通过非策略设置消融剪裁，验证其实际必要性。换言之，我们将采用基于单个标记的损失函数进行评估。

$$- \frac{\dot{U}_t(o_t | q, o_{<t})_A}{\pi_{\theta_{\text{old}}}(o_t | q, o_{<t})^t} \quad (34)$$

问题 (grpo_off_policy_clip_ablation)：离策略 GRPO -Clip消融 (2分) (2个H100小时)

交付成果：将未剪切的逐词损失实现为一种新的损失类型 “GRPO -No-Clip”。采用前一个问题中表现最佳的策略超参数，运行未剪切版本的损失函数。报告验证答案的奖励曲线。与 GRPO -Clip运行结果对比分析发现，包括熵、响应长度和梯度范数等具有显著趋势的其他指标。

提示的影响。作为最后的消融实验，我们将研究一个令人惊讶的现象：在强化学习 (RL) 过程中使用的提示，会根据模型的预训练方式，对模型性能产生显著影响。

我们不再使用 `cs336_alignment /prompts/ r1_zero .prompt` 中的 R1-Zero 提示，而是改用 `cs336_alignment /prompts/question_only.prompt` 中一个极其简单的提示：

```
{question}
```

您将使用此提示进行训练和验证，并将奖励函数（用于训练和验证）更改为位于 `cs336_alignment /drgrpo_grader` 中的 `question_only_reward_fn. py`。

问题 (grpo_prompt_ablation)：提示消融 (2分) (2 H100小时)

交付成果：报告 R1-Zero 提示和纯问题提示的验证答案奖励曲线。指标对比如何？包括熵值、响应长度和梯度范数等具有显著趋势的其他指标。请尝试解释研究结果。

9 排行榜: 数学 GRPO

作为（强制性）任务的最后一部分，您将在 2 块 H100 GPU 上进行实验，尝试在 4 小时训练时间内获取尽可能高的验证奖励。

模型。我们将继续使用 Qwen 2.5 Math 1.5B Base 模型。

数据集。我们将继续使用集群中提供的 MATH 训练和验证数据集（文件路径：`/data/a5-alignment/MATH/train.jsonl` 和 `/data/a5-alignment/MATH/validation.jsonl`）。严禁使用其他数据或对更强模型的推理链进行 SFT。必须使用上述采样超参数（温度 1.0，最大标记数 1024）报告整个验证集（全部 5000 个样本）的验证准确率。你可以自由筛选训练集，或根据需求设计基于数据的课程。验证时必须使用 R1-Zero 提示，且验证过程中必须严格采用启动代码中提供的 `r1_zero_reward_fn` 奖励函数（若需要，可自行开发其他奖励函数用于训练）。

算法。您可自由调整超参数或完全更换训练算法，但不得使用任何冗余数据或其他模型（如需，可自由使用更多模型副本）。

系统优化。你可能会注意到，在我们上面的简单 GRPO 实现中，至少有一个GPU始终处于空闲状态。通过优化流水线的系统特性，你很可能会获得显著提升。例如，可以考虑降低部署或训练的精度，使用torch编译器，以及其他系统优化措施。你完全不必将vLLM部署在单个设备上，而将训练策略部署在另一台设备上，我们鼓励你探索更好的并行化方案。

意见。如需了解可能的改进建议，请参阅以下资源：

- 静脉回路
- 拖车
- 调音
- 燕麦

关于KL散度。我们还注意到，在上述实验中，我们未包含相对于某些参考模型（通常为冻结 SFT 或预训练检查点）的KL散度项。在我们的实验及文献中的其他研究[Liu等人，2025，NTT123，2025]中，我们发现省略该项会导致

KL术语在节省GPU内存（无需存储参考模型）时对性能无影响。然而，许多 GRPO 的代码库默认包含该功能，并鼓励您尝试KL或其他形式的正则化，**只要您使用Qwen2.5Math1.5BBase或通过算法获得的模型即可。**

问题（排行榜）：排行榜（16分）（16 H100小时）

交付成果：报告在2个H100 GPU上训练4小时内获得的验证准确率，并提供验证准确率与实际运行时间的截图，其中x轴以0 4小时为终点。请注意，我们对您的评估设定了以下限制：

1. 您的验证准确率应为整个MATH验证集（全部5000个示例）的平均准确率。
2. 验证时必须使用R1-Zero提示。
3. 评估时必须使用温度1.0和最大1024个标记的vLLM。
4. 您需要通过计算 `r1_v` 零奖励函数（`zero_reward_fn`）在启动代码中生成的答案奖励的平均值来计算验证准确率。

10 尾声

恭喜你完成了本学期最后一项作业！你的努力值得骄傲。我们希望您通过从零开始构建现代语言模型的主要组件，能够愉快地掌握其底层原理。

参考文献

丹·亨德里克斯、科林·伯恩斯、绍拉夫·卡达瓦斯、阿库尔·阿罗拉、史蒂文·巴萨特、埃里克·唐、黎明·宋和雅各布·斯坦哈特。使用数学数据集测量数学问题解决能力，2021年。网址<https://arxiv.org/abs/2103.03874>。

DeepSeek-AI, 郭大亚, 杨德建, 张浩伟, 宋俊晓, 张若宇, 徐润欣, 朱启豪, 马世荣, 王佩仪, 毕晓, 张晓康, 余兴凯, 吴宇, Z. F. 吴, 苟志斌, 邵志宏, 李卓书, 高子怡, 刘爱欣, 薛冰, 王冰轩, 吴博超, 冯北, 卢成达, 赵成刚, 邓成启, 张晨宇, 阮冲, 戴大麦, 陈德利, 纪东杰, 李二航, 林芳云, 戴福聪, 罗福丽, 郝光波, 陈冠亭, 李国伟, 张浩, 韩宝, 徐汉伟, 王浩成, 丁红慧, 辛华建, 高华佐, 曲辉, 李辉, 郭建中, 李佳石, 王佳伟, 陈静昌, 袁静阳, 邱俊杰, 李俊龙, 蔡J. L., 倪佳琪, 梁建, 陈金, 董凯, 胡凯, 高凯格, 关康, 黄可欣, 快宇, 王廉, 张乐聪, 赵亮, 王立通, 张丽月, 徐磊, 夏雷毅, 张明川, 张明华, 唐明辉, 孟丽, 王妙军, 李明明, 宁天, 黄盼盼, 张鹏, 王千城, 陈琴玉, 杜秋实, 葛瑞奇, 张瑞松, 潘瑞哲, 王润吉, R. J. 陈, R. L. 金、陈如怡、卢尚浩、周尚艳、陈山煌、叶胜峰、王世宇、余水平、周顺峰、潘淑婷、李S.S.、周双、吴少清、叶胜峰、陶云、裴天、孙天宇、王T.、曾王定、赵万佳、刘文、梁文峰、高文军、余文琴、张文涛、肖W.L.、安伟、刘晓东、王晓涵、陈晓康、聂晓涛、程欣、刘欣、谢欣、刘兴超、杨新宇、李新元、苏学成、林旭恒、李X.Q.、金向月、沈晓金、陈晓沙、孙晓文、王晓翔、宋新南、周新毅、王贤祖、山新霞、李Y.K.、王Y.Q.、魏Y.X.、张阳、徐艳红、李瑶、赵瑶、孙耀峰、王耀辉、于毅、张一超、史一帆、熊一亮、何颖、刁一诗、王一松、谭一轩、马一阳、刘一元、郭永强、欧元、王玉丹、龚悦、邹玉恒、何玉佳、熊云帆、罗玉香、刘玉香、刘玉轩、周一阳、朱Y.X.、徐艳红、黄艳萍、李耀辉、郑毅、朱玉晨、马云贤、唐颖、查玉坤、严玉婷、Z. Z. 任泽辉、沙章立、付哲、徐哲安、谢振达、张正彦、郝哲文、马志诚、严志刚、吴志宇、顾子慧、朱子佳、刘子军、李子林、谢子威、宋子阳、潘子正、黄振、徐志鹏、张忠宇和张振。Deepseek-r1: 通过强化学习激励语言模型的推理能力, 2025。URL <https://arxiv.org/abs/2501.12948>。

Maxwell Nye、Anders Johan Andreassen、Guy Gur-Ari、Henryk Michalewski、Jacob Austin、David Bieber、David Dohan、Aitor Lewkowycz、Maarten Bosma、David Luan、Charles Sutton和Augustus Odena。展示你的工作: 语言模型中间计算的草稿, 2021年。网址<https://arxiv.org/abs/2112.00114>。

Jason Wei、Xuezhi Wang、Dale Schuurmans、Maarten Bosma、Brian Ichter、Fei Xia、Ed Chi、Quoc Le 和 Denny Zhou。思维链提示在大型语言模型中引发推理, 2023年。URL <https://arxiv.org/abs/2201.11903>。

Eric Zelikman、Yuhuai Wu、Jesse Mu 和 Noah D. Goodman。Star: Bootstrapping reasoning with reasoning, 2022。URL <https://arxiv.org/abs/2203.14465>。

Thomas Anthony、Zheng Tian 和 David Barber。利用深度学习和树搜索实现快速与慢速思考, 2017年。URL <https://arxiv.org/abs/1705.08439>。

OpenAI, :, 阿伦·杰奇, 亚当·卡莱, 亚当·勒勒, 亚当·理查森, 艾哈迈德·埃尔-基什基, 艾登·洛, 亚历克·赫利亚尔, 亚历山大·马德里, 亚历克斯·博伊特尔, 亚历克斯·卡尼, 亚历克斯·伊夫蒂米, 亚历克斯·卡尔彭科, 亚历克斯·塔查德·帕索斯, 亚历山大·内茨, 亚历山大·普罗科菲耶夫, 亚历山大·魏, 艾莉森·谭, 艾莉·贝内特, 阿娜雅·库马尔, 安德烈·萨拉伊瓦, 安德烈娅·瓦洛内, 安德鲁·杜伯斯坦, 安德鲁·孔德里奇, 安德烈·米申科, 安迪·阿普尔鲍姆, 安吉拉·蒋, 阿什文·奈尔, 巴雷特·佐夫, 贝鲁兹·戈尔巴尼, 本·罗森, 本杰明·索科洛夫斯基, 博阿兹·巴拉克, 鲍勃·麦格鲁, 鲍里斯·米纳耶夫, 高博涛, 博文·贝克, 布兰登·霍顿, 布兰登·麦金齐, 布莱登·伊斯曼, 卡米洛·卢加雷西, 卡里·巴斯因, 卡里·哈德森, 李查明, 查尔斯·德·布尔西, 切尔西·沃斯, 沈晨, 张冲, 克里斯·科赫, 克里斯·奥辛格, 克里斯托弗·赫塞, 克劳迪娅·菲舍尔, 克莱夫·陈, 丹·罗伯茨, 丹尼尔·卡普勒, 丹尼尔·莱维, 丹尼尔·塞尔萨姆, 大卫·多汉, 大卫·法尔希, 大卫·梅利, 大卫·罗宾

伊丽莎白·普罗尔、张恩科、埃里克·米切尔、埃里克·华莱士、埃里克·里特、埃文·梅斯、王凡、费利佩·佩特罗斯基·苏赫、菲利波·拉索、弗洛伦西亚·莱奥尼、福伊沃斯·钦普拉斯、弗朗西斯·宋、弗雷德·冯·洛曼、弗雷迪·苏利特、乔·萨尔蒙、吉安巴蒂斯塔·帕拉斯坎多洛、吉尔达斯·沙博特、赵格蕾丝、格雷格·布罗克曼、纪尧姆·勒克莱尔、哈迪·萨尔曼、包海明、盛浩、哈特·安德林、赫萨姆·巴赫里内扎德、任洪宇、亨特·莱特曼、郑亨元、伊恩·基维奇安、伊恩·奥康奈尔、伊恩·奥斯班德、伊格纳西·克拉韦拉·吉拉伯特、伊尔格·阿卡亚、伊利亚·科斯特里科夫、伊利亚·苏茨克韦尔、伊琳娜·科夫曼、雅库布·帕乔奇基、詹姆斯·伦农、韦杰森、让·哈布、杰里·特沃雷、冯佳成、余佳辉、翁佳毅、唐杰、余杰奇、华金·奎诺内罗·坎德拉、乔·帕勒莫、乔尔·帕里什、约翰内斯·海德克、约翰·霍尔曼、约翰·里佐、乔纳森·戈登、乔纳森·上田、乔纳森·沃德、尤斯特·胡伊辛加、王朱莉、陈凯、肖凯、卡兰·辛格尔、卡琳娜·阮、卡尔·科贝、石凯蒂、凯拉·伍德、肯德拉·林巴赫、克伦·古·伦伯格、刘凯文、卢凯文、卢凯文·斯通、卢凯文·余、拉玛·艾哈迈德、杨劳伦、刘雷奥、莱昂·马克辛、何立顿、利亚姆·费杜斯、温莉莲、李林登、林赛·麦卡勒姆、林赛·赫尔德、洛伦茨·库恩、卢卡斯·孔德拉丘克、卢卡什·凯撒、卢克·梅tz、玛德琳·博伊德、玛雅·特雷巴奇、马纳斯·约格莱卡尔、马克·陈、马尔科·廷托尔、梅森·迈耶、马特·琼斯、马特·考弗、马克斯·施瓦策、梅根·沙、梅赫梅特·亚特巴兹、梅洛迪·Y.关、徐梦远、严梦远、米娅·格莱斯、米安娜·陈、迈克尔·兰佩、迈克尔·马莱克、王米歇尔、米歇尔·弗拉丁、迈克·麦克莱、米哈伊尔·帕夫洛夫、王迈尔斯、王明轩、米拉·穆拉蒂、莫·巴伐利亚、穆斯塔法·罗哈尼内贾德、纳特·麦卡利斯、尼尔·乔杜里、尼尔·乔杜里、尼克·莱德、尼古拉斯·特扎克、诺亚姆·布朗、奥菲尔·纳胡姆、奥列格·博伊科、奥列格·穆尔克、奥利维亚·沃特金斯、帕特里克·赵、保罗·阿什伯恩、帕维尔·伊兹马伊洛夫、彼得·卓霍夫、蕾切尔·迪亚斯、拉胡尔·阿罗拉、兰德尔·林、拉法·贡蒂霍·洛佩斯、拉兹·高恩、瑞亚·米亚拉、雷马尔·莱克、雷尼·黄、节奏·加格、罗宾·布朗、罗山·詹姆斯、芮舒、瑞安·邱、瑞安·格林、萨奇·贾因、萨姆·阿尔特曼、萨姆·托伊泽、萨姆·托耶尔、塞缪尔·米塞伦迪诺、桑迪尼·阿加瓦尔、圣地亚哥·埃尔南德斯、萨沙·贝克、斯科特·麦金尼、斯科蒂·严、赵胜佳、胡胜利、希巴尼·桑图尔卡、沙拉曼·雷·乔杜里、张书远、傅思远、斯宾塞·帕佩、斯蒂芬·林、苏奇尔·巴拉吉、苏万什·桑杰夫、西蒙·西多尔、塔尔·布罗达、艾丹·克拉克、王涛、泰勒·戈登、泰德·桑德斯、特贾尔·帕特瓦尔丹、蒂博·索蒂亚克斯、托马斯·德格里、托马斯·迪姆森、郑天浩、蒂穆尔·加里波夫、汤姆·斯塔西、特拉皮特·班萨尔、特雷弗·克里奇、特洛伊·彼得森、蒂娜·埃隆杜、瓦莱丽·齐、维内特·科萨拉朱、文尼·莫纳科、维奇尔庞、弗拉德·福缅科、郑伟一、周文达、韦斯·麦凯布、沃伊切赫·扎伦巴、扬·杜布瓦、陆英海、陈一宁、查永、白宇、何玉晨、张玉晨、王云云、邵正和李卓汉。OpenAI O1系统卡片，2024年。URL <https://arxiv.org/abs/2412.16720>。

Kimi团队、杜安刚、高博飞、邢宝伟、蒋长久、陈晨、陈丽、肖晨俊、杜晨庄、廖崇华、唐春宁、王聪聪、张德浩、袁恩明、卢恩哲、唐凤祥、宋洪水、魏广达、赖国坤、郭海青、朱汉、丁浩、胡浩、杨浩、张浩、姚浩天、赵浩天、卢浩宇、李浩泽、余浩臻、高洪成、郑华斌、袁欢、陈佳、郭建航、苏建林、王建洲、赵杰、张金、刘景远、严俊杰、吴俊艳、史立东、叶玲、余龙辉、董梦楠、张新奥、马宁辰、潘启伟、龚秋成、刘少伟、马胜玲、魏书鹏、曹思翰、黄思颖、蒋涛、高伟豪、熊伟民、何伟然、黄伟晓、吴文豪、何文扬、魏向辉、贾贤庆、吴兴哲、徐欣然、徐欣星、周新宇、潘学海、Y. Charles、李阳、胡阳阳、刘阳阳、陈燕如、王烨杰、刘一博、秦一达、刘一峰、杨颖、包一平、杜玉伦、吴宇欣、王玉芝、周大达、王兆基、李昭伟、朱振、张正、王泽旭、杨志林、黄志奇、黄子豪、徐子耀和杨宗翰。Kimi k1.5：利用llms扩展强化学习，2025年。URL <https://arxiv.org/abs/2501.12599>。

Richard S Sutton、David McAllester、Satinder Singh和Yishay Mansour。基于函数逼近的强化学习策略梯度方法。见S. Solla、T. Leen和K. Muller主编的《神经信息处理系统进展》第12卷。MIT Press，1999年。URL https://proceedings.neurips.cc/paper_files/paper/1999/file/464d828b85b0bed98e80ade0a5c43b0f-Paper.pdf。

Hugging Face. Open r1: DeepSeek-r1 的完全公开版本，2025年1月。URL <https://github.com/huggingface/open-r1>。

曾卫豪、黄玉珍、刘倩、刘伟、何克清、马泽军、何俊贤。Simplerl-zoo:

研究和驯化野外开放基模型的零强化学习，2025年。URL <https://arxiv.org/abs/2503.18892>。

潘佳怡、张俊杰、王星瑶、袁立凡、彭浩和苏尔·阿兰。Tinyzero。 <https://github.com/Jiayi-Pan/TinyZero>，2025年。访问日期：2025年1月24日。

杨阳、张柏辰、惠斌远、高博飞、余博文、李成鹏、刘大一恒、涂建宏、周景仁、林俊阳、卢克明、薛明峰、林润基、刘天宇、任兴章、张振儒。Qwen2.5-math技术报告：通过自我改进实现数学专家模型，2024。URL <https://arxiv.org/abs/2409.12122>。

卡尔·科贝、维内特·科萨拉朱、穆罕默德·巴伐利亚、陈马克、俊熙宇、卢卡什·凯撒、马蒂亚斯·普拉佩特、杰里·特沃雷克、雅各布·希尔顿、中野亮一郎、克里斯托弗·赫塞和约翰·舒尔曼。培训验证者解决数学应用题，2021a。网址<https://arxiv.org/abs/2110.14168>。

内森·兰伯特、雅各布·莫里森、瓦伦蒂娜·皮亚特金、黄胜义、哈米什·艾维森、法伊兹·布拉曼、莱斯特·詹姆斯·V·米兰达、阿丽莎·刘、努哈·德齐里、刘善、顾玉玲、索米娅·马利克、维多利亚·格拉夫、黄珍娜·D·黄、杨江江、罗南·勒布拉斯、奥伊文德·塔夫约德、克里斯·威廉姆、卢卡·索达伊尼、诺亚·A·史密斯、王一中、普拉迪普·达西吉和哈纳内·哈吉希尔齐。图卢3：开放语言的前沿探索模型训练后，2025年。网址<https://arxiv.org/abs/2411.15124>。

刘子晨、陈长宇、李文军、齐鹏辉、庞天宇、杜超、孙伟李和林敏。理解r1-zero类训练：一个关键视角，2025。URL <https://arxiv.org/abs/2503.20783>。

吴素坤、李卓翰、庄思远、盛颖、郑连民、郝科迪、约瑟夫·E·冈萨雷斯、张浩和伊恩·斯托伊卡。基于分页注意力机制的大语言模型高效内存管理，2023年。arXiv:2309.06180。

卡尔·科贝、维内特·科萨拉朱、穆罕默德·巴伐利亚、陈马克、俊熙宇、卢卡什·凯撒、马蒂亚斯·普拉佩特、杰里·特沃雷克、雅各布·希尔顿、中野亮一郎、克里斯托弗·赫塞和约翰·舒尔曼。培训验证者解决数学应用题，2021b。arXiv:2110.14168。

David Dohan、Winnie Xu、Aitor Lewkowycz、Jacob Austin、David Bieber、Raphael Gontijo Lopes、Yuhuai Wu、Henryk Michalewski、Rif A. Saurous、Jascha Sohl-dickstein、Kevin Murphy和Charles Sutton。语言模型级联，2022年。网址<https://arxiv.org/abs/2207.10342>。

Caglar Gulcehre、Tom Le Paine、Srivatsan Srinivasan、Ksenia Konyushkova、Lotte Weerts、Abhishek Sharma、Aditya Siddhant、Alex Ahern、Miaosen Wang、Chenjie Gu、Wolfgang Macherey、Arnaud Doucet、Orhan Firat和Nando de Freitas。强化自我训练（休息）用于语言建模，2023年。URL <https://arxiv.org/abs/2308.08998>。

Joshua Achiam。深度强化学习中的自旋加速现象。2018a。

Nathan Lambert。人类反馈强化学习，2024。URL <https://rlhfbook.com>。Sheldon M Ross. *Simulation*. academic press, 2022。

Thomas Degris、Martha White 和 Richard S. Sutton。非策略性演员-评论家，2013年。URL <https://arxiv.org/abs/1205.4839>。

邵志宏、王佩怡、朱启豪、徐润欣、宋俊晓、毕晓、张浩伟、张明川、李Y.K.、吴Y.和郭大亚。Deepseek-math：突破数学推理的极限
开放语言模型，2024年。网址<https://arxiv.org/abs/2402.03300>。

约翰·舒尔曼、菲利普·沃尔斯基、普拉富拉·达里瓦尔、亚历克·拉德福德和奥列格·克利莫夫。近端策略优化算法，2017年。网址<https://arxiv.org/abs/1707.06347>。

Joshua Achiam。简化版 ppo-clip 目标，2018b。网址 <https://drive.google.com/file/d/1PDzn9RPvaXjJFZkGeapMHbHGjWW20Ey/view>。

余启颖、张正、朱若飞、袁宇峰、左晓晨、岳宇、范天天、刘高宏、刘凌军、刘欣、林海斌、林志奇、马伯乐、盛光明、童宇轩、张驰、张莫凡、张望、朱航、朱金华、陈佳泽、陈江杰、王成毅、弘历宇、戴文南、宋宇轩、魏向鹏、周浩、刘静静、马伟英、张雅琴、林燕、缪乔、吴永辉和王明轩。Dapo: 一个开源的llm强化学习大规模系统，2025年。网址<https://arxiv.org/abs/2503.14476>。

NTT123. 零群。<https://github.com/policy-gradient/GRPO-Zero>，2025年。访问日期：2025年5月22日。